

FPGA TABANLI FONKSİYON ÜRETECİ

2024

LİSANS TEZİ

ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ

Muzaffer KARAMAN

Salim Enes ALTURAN

FPGA TABANLI FONKSİYON ÜRETECİ

Muzaffer KARAMAN

Salim Enes ALTURAN

Karabük Üniversitesi

Mühendislik Fakültesi

Elektrik-Elektronik Mühendisliği Bölümünde

Lisans Tezi

Olarak Hazırlanmıştır

KARABÜK

Haziran 2024

Salim Enes ALTURAN ve Muzaffer KARAMAN tarafından hazırlanan “FPGA Tabanlı Fonksiyon Üretici” başlıklı bu tezin Lisans Tezi olarak uygun olduğunu onaylarım.

11/06/2024

Doç. Dr. Ahmet Hayrettin YÜZER

.....

Tez Danışmanı,

Karabük Üniversitesi Elektrik-Elektronik Mühendisliği Bölümü

ÖZET

Lisans Tezi

FPGA TABANLI FONKSİYON ÜRETECİ

Muzaffer KARAMAN

Salim Enes ALTURAN

Karabük Üniversitesi

Mühendislik Fakültesi Elektrik-Elektronik Mühendisliği Bölümü

Tez Danışmanı

Doç. Dr. Ahmet Hayrettin YÜZER

Haziran 2024, 69 sayfa

Bu çalışmada, FPGA kullanılarak istenilen frekansta ve yüksek frekans çözünürlüğünde dalga formları üretebilen bir fonksiyon üretici tasarlanmıştır. 1 MHz ve üzerinde frekans değeri elde edilmesi hedeflenmiştir. FPGA'ler bir donanım tanımlama dili kullanılması ile sayısal tasarım yapılmasını sağlayan entegre devrelerdir. Günümüzde tasarımda büyük esneklik sağlaması ve paralel işlem yeteneği sebebi ile FPGA'ler oldukça yaygınlaşmıştır. FPGA ile donanım tasarımı yapılabilmesi amacıyla geliştirilmiş birçok dil bulunmaktadır. Bunlardan yaygın olanları VHDL ve Verilog'dur. Bu çalışmada VHDL dili kullanılmıştır.

Bu çalışmada sinyal katsayıları bilgisayar üzerinden FPGA'ye aktarılmıştır. FPGA üzerinden de DAC'a aktararak analog sinyal elde edilmiştir. Bilgisayardan FPGA'ye veri aktarımında UART Protokolü kullanılmıştır. FPGA'den DAC'a verilerin aktarılmasında ise SPI Protokolü kullanılmıştır.

Anahtar Kelimeler : FPGA, UART, SPI, DAC, VHDL, Fonksiyon Üretici

ABSTRACT

B. Sc. Thesis

FPGA BASED FUNCTION GENERATOR

Muzaffer KARAMAN

Salim Enes ALTURAN

Karabük University

Faculty of Engineering Department of Electrical-Electronics Engineering

Thesis Advisor

Doç. Dr. Ahmet Hayrettin YÜZER

June 2024, 69 pages

In this study, a function generator that can generate waveforms at the desired frequency and high frequency resolution using FPGA was designed. It is aimed to obtain a frequency value of 1 MHz and above. FPGAs are integrated circuits that enable digital design using a hardware description language. Nowadays, FPGAs have become quite common due to their great flexibility in design and parallel processing capability. There are many languages developed to design hardware with FPGA. Common of these are VHDL and Verilog. VHDL language was used in this study.

In this study, the signal coefficients will be transferred to the FPGA via the computer. An analog signal was obtained by transferring it to the DAC via FPGA. UART Protocol is used to transfer data from the computer to the FPGA. SPI Protocol was used to transfer data from FPGA to DAC.

Key Words : FPGA, UART, SPI, DAC, VHDL, Function Generator

TEŞEKKÜR

Bu çalışmada değerli bilgi ve yardımlarıyla destek olan, bilgi ve tecrübeleri ile bizlere ışık olan, bilgilendirme ve yönlendirmeleri ile çalışmamızı şekillendiren değerli danışman hocamız Doç. Dr. Ahmet Hayrettin YÜZER’e teşekkürlerimizi sunarız.

Her zaman yanımızda olan, maddi manevi desteklerini esirgemeyen ailemize, tez süresince bilgi ve birikimlerinden yararlandığımız meslektaş arkadaşlarımıza teşekkürlerimiz sunarız.

“Bu tezdeki tüm bilgilerin akademik kurallara ve etik ilkelere uygun olarak elde edildiğini ve sunulduğunu; ayrıca bu kuralların ve ilkelerin gerektirdiği şekilde, bu çalışmadan kaynaklanmayan bütün atıfları yaptığımı beyan ederim.”

Muzaffer KARAMAN

Salim Enes ALTURAN

İÇİNDEKİLER

Sayfa

ÖZET	iii
ABSTRACT	iv
TEŞEKKÜR	v
İÇİNDEKİLER.....	vi
ÇİZELGELERİN LİSTESİ	ix
ŞEKİLLERİN LİSTESİ.....	x
BÖLÜM 1.....	1
1. GİRİŞ	1
BÖLÜM 2.....	2
2. FPGA (FIELD PROGRAMMABLE GATE ARRAY).....	2
2.1. FPGA Tarihçesi	2
2.2. FPGA Nedir	3
2.3. FPGA Mimarisi ve Pinleri	3
2.4. FPGA Avantajları	5
2.5. FPGA Programlama.....	5
2.6. Donanım Tanımlama Dilleri	6
2.6.1. VHDL	6
2.6.2. Verilog	6
2.7. FPGA Üreticileri.....	6
BÖLÜM 3.....	8
3. PROJE KAPSAMINDA KULLANILAN DONANIMLAR.....	8
3.1. BASYS3.....	8
3.1.1. Pmod Konnektörleri.....	10
3.2. PMOD DA3	11

BÖLÜM 4.....	14
4. PROJE KAPSAMINDA KULLANILAN UYGULAMALAR.....	14
4.1. Xilinx Vivado.....	14
4.2. Matlab	16
4.3. Notepad++	17
4.4. Hterm	18
BÖLÜM 5.....	19
5. PROJE KAPSAMINDA YAPILAN İŞLEMLER.....	19
5.1. Notepad++ ile Yazılım Oluşturma.....	19
5.2. Vivado Uygulaması	20
5.2.1. Vivado Proje Oluşturma	20
5.2.2. Vivado’da Simülasyon ile Tasarımın Doğrulanması.....	23
5.2.3. Vivado ile Oluşturulan Projenin FPGA’e Yüklenmesi.....	25
5.3. Matlab Uygulaması.....	25
5.4. Hterm Uygulaması.....	26
5.5. Haberleşme Protokolü.....	27
5.5.1. UART (Universal asynchronous receiver/transmitter) Protokolü	27
5.5.2. SPI (Serial Peripheral Interface) Protokolü	29
5.6. Üst Modül Oluşturulması.....	32
BÖLÜM 6.....	33
6. PROJENİN KULLANIMI VE SİNYAL OLUŞTURULMASI	33
6.1. Matlab’da İstenilen Sinyal Katsayılarının Üretilmesi.....	33
6.2. FPGA (Basys3) Kartının Programlanması.....	34
6.3. Hterm ile Veri Aktarımı.....	34
6.4. Farklı Sinyal Formlarının Üretilmesi.....	34
6.4.1. Sinüs Dalga Formu	34
6.4.2. Kare Dalga Formu.....	36

6.4.3. Üçgen Dalga Formu.....	38
6.4.4. Testere Dişı Dalga Formu.....	40
BÖLÜM 7.....	43
7. SONUÇ VE ÖNERİLER.....	43
KAYNAKLAR.....	44
EKLER	46
EK 1. UART_RX Modülü	46
EK 2. SPI Modülü	49
EK 3. Üst Modül	52
EK 4. Matlab Yazılımı	57
EK 5. SPI ile Veri Gönderme Modülü	60
EK 6. TestBench Yazılımı.....	63
ÖZGEÇMİŞ.....	69

ÇİZELGELERİN LİSTESİ

Çizelge	Sayfa
Tablo 1. Basys3 Kartı Çevresel Birim Açıklamaları.....	10
Tablo 2. Pmod DA3 Pinleri	12

ŞEKİLLERİN LİSTESİ

Şekil	Sayfa
Şekil 2.1 Teknolojilerin zaman içindeki gelişimi.....	2
Şekil 2.2 FPGA Mimarisi	3
Şekil 2.3 Mantık Hücresi	4
Şekil 2.4 FPGA Tasarım Süreci	5
Şekil 3.1 Basys3	8
Şekil 3.2 Basys3 Kartı Çevresel Birimler.....	9
Şekil 3.3 Pmod Konnektörü	10
Şekil 3.4 Basys3 Pmod Konnektörleri.....	11
Şekil 3.5 Pmod DA3	11
Şekil 4.1 Tasarım Metodolojisi	14
Şekil 4.2 Sentez Örneği	15
Şekil 4.3 IP Entegrasyonu	16
Şekil 5.1 Notepad++ Uygulama Ekranı.....	19
Şekil 5.2 Notepad++ Oluşturulan Yazılımın Kaydedilmesi.....	20
Şekil 5.3 Vivado Proje Oluşturma Ekranı	21
Şekil 5.4 Vivado Proje Kaynak Dosyası Ekleme	22
Şekil 5.5 Vivado FPGA Constraint Dosyası Ekleme	22
Şekil 5.6 Vivado FPGA Bord Seçimi.....	23
Şekil 5.7 Vivado Proje Simülasyon Dosyası Ekleme.....	24
Şekil 5.8 Vivado Proje Simülasyonu.....	24
Şekil 5.9 Matlab Uygulama Ekranı	26
Şekil 5.10 UART Receiver.....	27
Şekil 5.11 Uart Haberleşmesi ile Hterm Üzerinden Veri Gönderimi.....	28
Şekil 5.12 Gönderilen Verilerin FPGA üzerindeki Ledlerde Gösterimi	29

Şekil 5.13 Clock Divider	30
Şekil 5.14 PmodDA3 Zaman Diyagramı.....	31
Şekil 5.15 Sinüs Dalga Formu.....	32
Şekil 6.1 Matlab'da Katsayı Oluşturulması	33
Şekil 6.2 100 Adet Veri ile Sinüs Dalga Formu.....	34
Şekil 6.3 10 Adet Veri ile Sinüs Dalga Formu.....	35
Şekil 6.4 4 Adet Veri ile Sinüs Dalga Formu.....	35
Şekil 6.5 2 Adet Veri ile Sinüs Dalga Formu.....	36
Şekil 6.6 100 Adet Veri ile Kare Dalga Formu	36
Şekil 6.7 10 Adet Veri ile Kare Dalga Formu	37
Şekil 6.8 4 Adet Veri ile Kare Dalga Formu	37
Şekil 6.9 2 Adet Veri ile Kare Dalga Formu	38
Şekil 6.10 100 Adet Veri ile Üçgen Dalga Formu	38
Şekil 6.11 10 Adet Veri ile Üçgen Dalga Formu	39
Şekil 6.12 4 Adet Veri ile Üçgen Dalga Formu	39
Şekil 6.13 2 Adet Veri ile Üçgen Dalga Formu	40
Şekil 6.14 100 Adet Veri ile Testere Dişi Dalga Formu	40
Şekil 6.15 10 Adet Veri ile Testere Dişi Dalga Formu	41
Şekil 6.16 4 Adet Veri ile Testere Dişi Dalga Formu	41
Şekil 6.17 2 Adet Veri ile Testere Dişi Dalga Formu	42

BÖLÜM 1

1. GİRİŞ

Günümüzde, sinyal işleme uygulamaları hayatımızın her alanında önemli bir rol oynamaktadır. İleri teknoloji ürünlerinden sağlık sektörüne, iletişimden otomasyona kadar birçok alanda sinyal işleme algoritmaları kullanılmaktadır. Bu algoritmalar, çeşitli sinyallerin (ses, görüntü, sensör verileri vb.) işlenmesini, analiz edilmesini ve yorumlanmasını sağlayarak, bilgiye dönüşüm sürecinde kritik bir rol oynamaktadır.

FPGA (Field Programmable Gate Array), son yıllarda sinyal işleme uygulamalarında giderek artan bir popülerlik kazanmıştır. Bu esnek entegre devreler, özelleştirilebilir mimarileri ve paralel işlem yetenekleri sayesinde sinyal işleme algoritmalarının yüksek performanslı ve hızlı bir şekilde uygulanmasına imkân tanır.

FPGA tabanlı sinyal işleme uygulamaları, geleneksel mikroişlemci veya DSP tabanlı sistemlere göre birçok avantaj sunar. Bunlar arasında düşük gecikme, paralel işleme yetenekleri, enerji verimliliği ve yüksek esneklik sayılabilir.

Bu tez çalışmasında da FPGA tabanlı bir fonksiyon üretici tasarlanmıştır. Bilgisayar tarafından gönderilen veriler FPGA borduna ulaşmakta, FPGA'den de DAC'a ulaşarak istenilen formda sinyaller üretilebilmektedir.

Tez kapsamında öncelikli olarak kullanılan donanımlar, uygulamalar ve yöntemlerin tanımlarına yer verilmiştir. Sonraki bölümlerde projenin yapım aşamalarına ve kullanımına değinilmektedir.

BÖLÜM 2

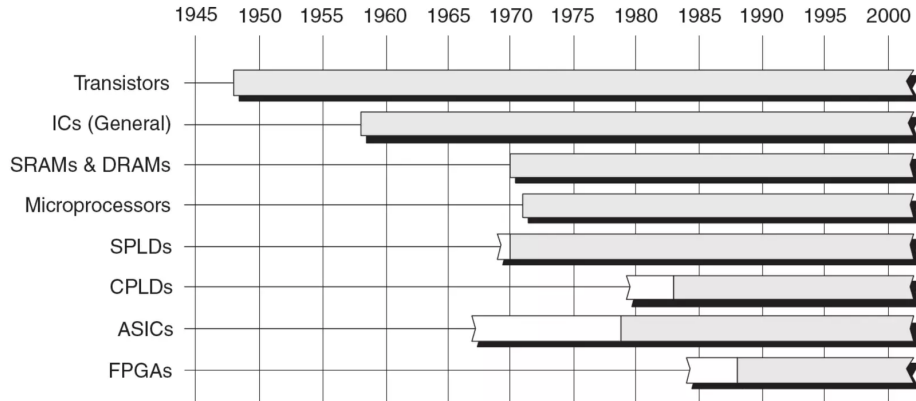
2. FPGA (FIELD PROGRAMMABLE GATE ARRAY)

Bu bölümde FPGA (Alan Programlanabilir Kapı Dizisi) ayrıntılı biçimde incelenmiştir. FPGA tarihçesi, ne amaçla kullanıldığı, mimarisi, çalışma prensibi, avantajları ve neden tercih edildiği gibi konulara değinilmiştir.

2.1. FPGA Tarihçesi

FPGA'ler ilk olarak 1985 yılında ortaya çıkmaktadır. FPGA'lerden öncesinde PLA denilen Programlanabilir Lojik Kapılar bulunmaktadır. Şekil 2.1'de teknolojilerin yıllar içerisindeki oluşumları ve gelişimleri verilmiştir. Çubuklarda bulunan gri bölgeler ürünün yaygın kullanıldığı ve kabul gördüğü zamanları ifade etmektedir. FPGA'ye karşı rağbet 1990'lı yılların başlarında doğru başlamıştır. [13]

Technology Timeline



Şekil 2.1 Teknolojilerin zaman içindeki gelişimi

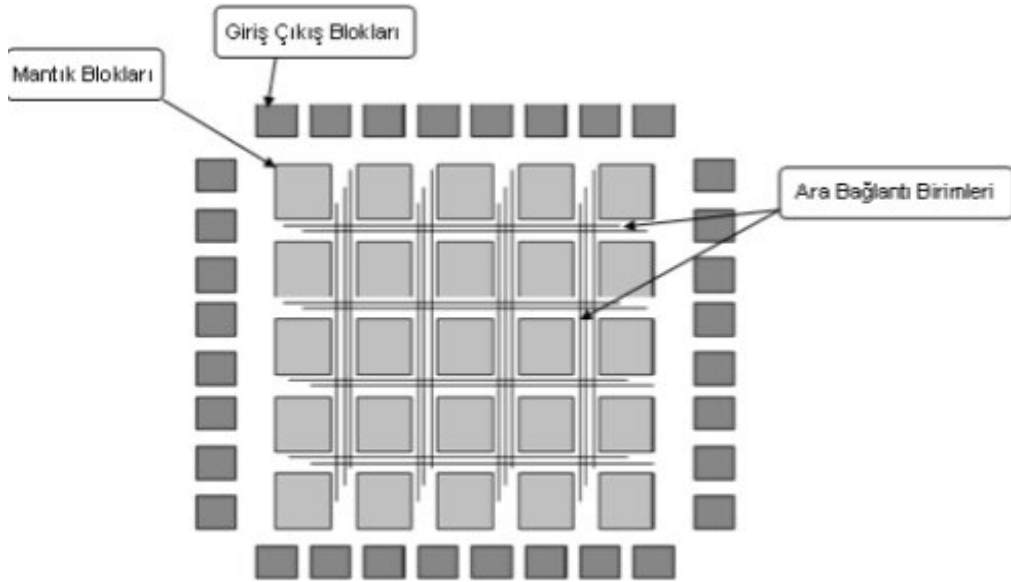
1990'lı yılların başında artan kapasiteleri, geniş veri işlemleri gerektiren haberleşme ve ağ ortamlarında kullanımları artmıştır.

2.2. FPGA Nedir

FPGA (Field Programmable Gate Array), içerisinde programlanabilir pek çok mantık devresi barındıran tümleşik devredir. Bu mantık devreleri ile yapılmak istenilen tasarıma göre birbirlerine bağlanarak düzenlenebilmekte ve bu sayede pek çok donanımın gerçekleştirilmesi mümkün olmaktadır. [13][8]

2.3. FPGA Mimarisi ve Pinleri

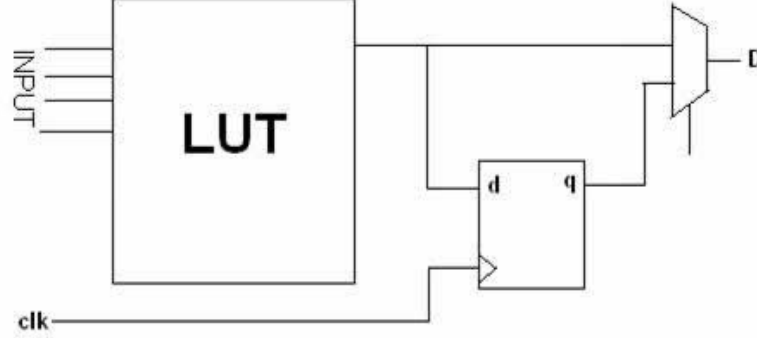
FPGA; Mantık Hücreleri (Logic Cell), Giriş/Çıkış Bloklarından (IO Block) ve Ara bağlantılardan meydana gelen bir yapıdır. Şekil 2.2.'de FPGA Mimarisi bulunmaktadır.



Şekil 2.2 FPGA Mimarisi

FPGA'in ana yapısını Mantık Hücreleri oluşturmaktadır. Bir Logic-Cell 1 adet Lookup Table (LUT), 1 adet D Flip-Flop ve bir adet 2 to 1 Mux' tan oluşmaktadır. LUT 'lar bir mantık işlemi yerine getiren küçük belleklerdir (RAM). N girişli bir LUT, 2^N 'li bir belleğe işaret etmektedir. Binlerce Mantık Hücrelerinin birleşimi sonucunda kompleks ve büyük programlar oluşturulmaktadır. Matris şeklindeki veri yolları ve programlanabilir anahtarlarla (FPGA yüklenen programa göre) Mantık Hücrelerinin ara bağlantıları sağlanmaktadır. FPGA tasarımı ile mantık hücrelerinin uygulayacağı fonksiyonlar ve programlanabilir

anahtarların durumu (açık/kapalı) belirlenerek mantık hücreleri arasındaki bağlantılar tanımlanmaktadır. Şekil 2.3'te Mantık Hücresi yapısı bulunmaktadır.[8]



Şekil 2.3 Mantık Hücresi

FPGA pinleri 2 kategoriye ayrılmaktadır:

- a. Ayrılmış pinler (Dedicated pins)
- b. Kullanıcı pinleri (User pins)

- a) Ayrılmış pinler: Bir FPGA'de pinlerin yaklaşık %20 ila %30'u ayrılmış pindir. FPGA'de gerçekleştirdikleri özel fonksiyonlara göre bu pinler 3'e ayrılır:
 - 1. Güç Pinleri: FPGA için gerekli olan güç ve ground (Toprak) sağlayan pinlerdir.
 - 2. Konfigürasyon Pinleri: Oluşturulan programın FPGA yüklenmesi (download) için kullanılan pinlerdir.
 - 3. Clock Pinleri: Clock sinyalleri için ayrılmış özel pinlerdir.
- b) Kullanıcı Pinleri: Bunlar kullanıcı tarafından tasarlanabilen standart I/O pinleridir. Input, Output, Input/Output olarak üç kategoriye ayrılmaktadır. Input, Output, Input/Output olarak üç bölüme ayrılmaktadır. FPGA'de her bir I/O pini, bir I/O hücresine bağlıdır. VCCIO ile I/O hücrelerinin güçleri sağlanmış olur. Eski FPGA'ler birden fazla VCCIO pini olduğu halde, bütün pinler aynı voltajla beslenmekteydi. Yeni üretilen FPGA'lerde ise I/O'lar gruplara ayrılabilir ve bu gruplar farklı voltajlardan beslenebilirler. Böylelikle bir grup IO pinleri 3.5 volt ile çalışırken; diğer grup I/O pinleri de 2.5 volt ile çalışabilmektedir.

2.4. FPGA Avantajları

FPGA'ler yüksek oranda paralel veri işleme olanağı sağlamaktadır. Özellikle görüntü işleme, sinyal işleme vb. yoğun bilgi içeren uygulamalarda veriyi kısa sürede işleyebilmektedir.

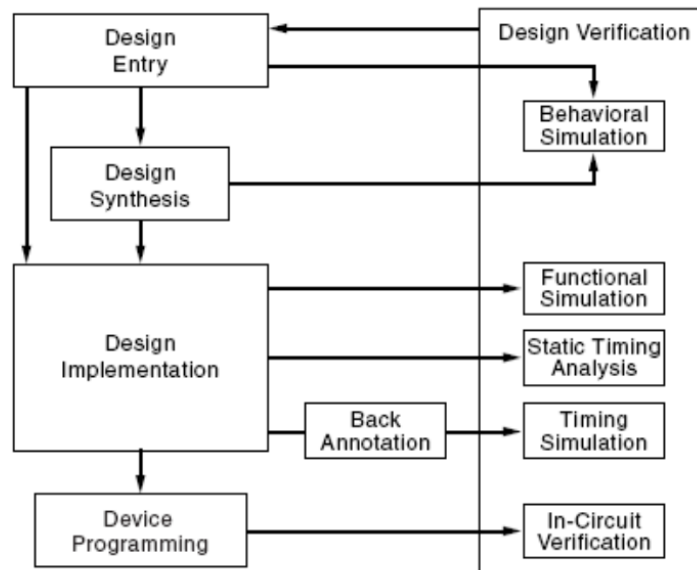
Pek çok birimin eş zamanlı çalışabilmesi sayesinde tüm sistemin geleneksel bilgisayar mimarisine göre daha düşük frekanslarda çalışabilmesine olanak sağlamaktadır.

Ayrıca FPGA'ler düşük güç tüketmesi ile de avantaj sağlamaktadır. Enerji kaynaklarının kısıtlı olduğu, gerçek zamana yakın biçimde işlenmesi gerekli olduğu ortamlarda iyi bir seçenek olmaktadır. [13]

2.5. FPGA Programlama

Tasarımcı tarafından donanım tanımlama dili (VHDL ya da Verilog) belirlenerek FPGA ile uygulaması yapılmak istenilen devre tasarlanır. Bilgisayar destekli tasarım yazılımı yardımıyla, yazılan kod sentezlenir. Diğer bir bilgisayar destekli tasarım yazılımı kullanılarak sentezlenen koda karşılık uygun bağlantılar FPGA içerisinde gerçekleştirilir. Yazılımın gerekli veri akışı oluşturulur ve FPGA içerisine aktarılarak FPGA programlanır.

Şekil 2.4'te FPGA Tasarım Süreci belirtilmiştir.[13]



Şekil 2.4 FPGA Tasarım Süreci

2.6. Donanım Tanımlama Dilleri

2.6.1. VHDL

VHDL sayısal elektronik sistemleri tanımlamak ve bir FPGA içine gömmek için kullanılan bir programlama dilidir. İlk geliştirilme çalışmaları 1980’li yıllara dayanmaktadır. VHDL’in geliştirilmesindeki amaç modelleme ve simülasyondur. Ancak sonradan tasarım aşamalarının otomasyonunu sağlayabilmek için sentezlemede de kullanılmaya başlanmıştır. VHDL, Verilog’a göre daha esnek tasarımlara imkân vermekle birlikte üst seviyeli tasarımlara daha uygundur. [13]

2.6.2. Verilog

Verilog, 1980’li yıllarda geliştirilmeye başlanmış sayısal elektronik sistemleri tanımlamak ve bir FPGA içine gömmek için kullanılan bir programlama dilidir. Verilog dilinde devreler kapı seviyesinde modellenebileceği gibi davranışsal olarak da modellenabilmektedir. Davranışsal modelleme, sentez yapacak olan yazılım aracına nasıl bir devre istendiğinin tarif edilmesidir, kapı seviyesinde ayrıntı vermeye gerek kalmadan yapılan tasarımıdır.

2.7. FPGA Üreticileri

Xilinx ve Altera dünyadaki FPGA pazarının en büyük iki üreticileridir.

- Xilinx: FPGA’in mucidi olan firmadır. FPGA pazarının en büyük firmasıdır. Tasarım ve sentez için ISE programını sunmaktadır. Artix, Kintex Spartan bu firmanın sunduğu FPGA ürün serilerindendir.



- Altera: Xilinx firmasının en büyük rakibidir. Tasarım ve sentezi için Quartus II programını sunmaktadır. Stratix, Cyclone, Arria bu firmanın sunduğu FPGA serileridir.



- Lattice, Actel, Quicklogic ve SiliconBlue firmaları ise diğer FPGA üreticileridir. Xilinx ve Altera dünyadaki FPGA pazarının iki en büyük üreticileridir. [8]

BÖLÜM 3

3. PROJE KAPSAMINDA KULLANILAN DONANIMLAR

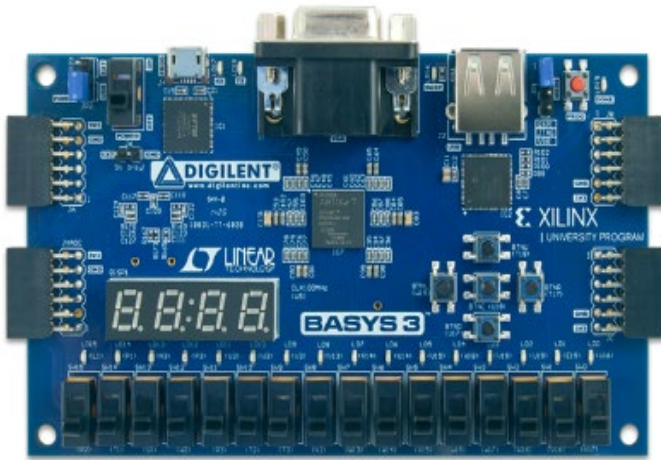
Bu projede donanım olarak Basys3 ve PmodDA3 (digital-analog converter) kullanılmıştır. Bu bölümde proje kapsamında kullanılan donanımlar ayrıntılı olarak anlatılmıştır.

3.1. BASYS3

Basys3, son model Artix-7 FPGA kullanılarak üretilen kullanıma hazır bir devre geliştirme platformudur. Yüksek kapasiteli alanda programlanabilir kapı dizileri (FPGA), düşük maliyetli, USB, VGA ve portların birleşmesiyle, Basys3, giriş niteliğindeki devrelerden karışık dizi halindeki devrelere kadar birçok devreyi gömülü işlemciler ve denetleyiciler gibi çalışmaya olanak sağlamaktadır. İçinde yeteri kadar anahtar, Led ve giriş/çıkış donanımı bulunan Basys3, herhangi bir ek donanıma ihtiyaç duymadan birçok tasarımı eksiksiz çalıştırabilme imkânı tanımaktadır. Ayrıca kart dışını çıkarılmış giriş ve çıkış portları, yapılan tasarımların Digilent Pmod'lar veya diğer özel devre kartları ve devrelerle genişletilmesine izin vermektedir.

Artix-7 FPGA yüksek performans için optimize edilmiştir. Önceki tasarımlara göre daha fazla kapasite, daha yüksek performans ve daha fazla kaynak sunmaktadır.

Artix-7 35T'nin sahip olduğu özellikler aşağıdaki gibidir.



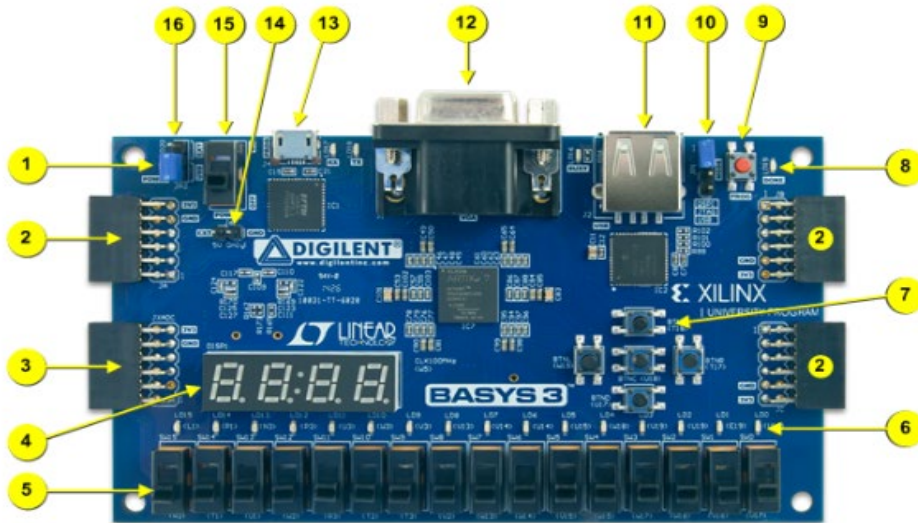
Şekil 3.1 Basys3

- 5200 dilimde 33.280 mantık hücresi (her dilim 4-6 girişli LUT ve 8 flip-flop içeriyor.)
- 1,800 Kbit hızlı blok RAM;
- Her biri PLL'e sahip, beş adet clock yönetim modülü;
- 90 DSP dilim;
- Dahili 450 MHz'i aşan saat hızları;
- Analog-Digital çevirici (XADC)

Ayrıca Basys3 zengin ve geliştirilmiş çevresel donanım yelpazesi sunmaktadır. Bunlardan bazıları aşağıda sıralanmıştır.

- 16 Adet Anahtar
- 16 adet LED
- 5 adet buton
- 4 haneli 7 parçalı gösterge
- 3 adet Pmod konektörü
- XADC için Pmod konektörü
- 12 bitlik VGA çıkışı
- USB-UART köprüsü
- Ses harici flash hafıza
- FPGA programlama ve haberleşme için Digilent USB-JTAG portu
- Flaş disk, fare ve klavye için USB HID sunucusu

Basys3, Xilinx'in yeni yüksek performanslı programı Vivado Design Suit ile çalışmaktadır. Vivado, güncel tasarım metotlarını kolaylaştıran ve güçlendiren birçok yeni araç ve tasarım akışı içermektedir. Daha hızlı çalışmaktadır, FPGA kaynaklarını daha verimli kullanmamıza olanak sağlamakta ve proje üreticilerine tasarım alternatiflerini deneme imkânı vermektedir. Vivado'nun "The System Edition" sürümü yongayla tümleşik bir lojik çözümler, yüksek seviyede bir sentez donanımı sağlamaktadır. Şekil 3.2'de Basys3 çevresel birimleri numaralandırılmıştır.[7]



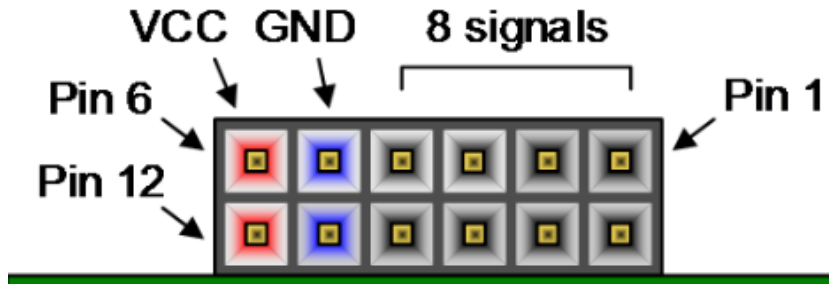
Şekil 3.2 Basys3 Kartı Çevresel Birimler

Tablo 1. Basys3 Kartı Çevresel Birim Açıklamaları

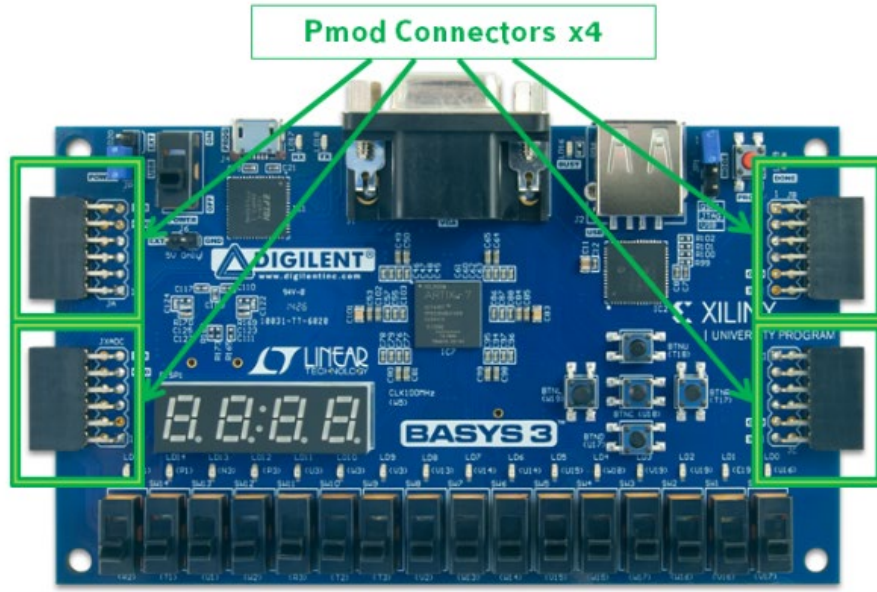
No	Bileşen Açıklaması	No	Bileşen Açıklaması
1	Güç Led'i	9	FPGA yapılandırması reset butonu
2	Pmod Konnektörleri	10	Programlama modu Jumper'ı
3	Analog Sinyal Pmod Konnektörü (XADC)	11	USB sunucu bağlayıcısı
4	4 haneli 7 parçalı LED gösterge	12	VGA bağlayıcısı
5	Anahtarlar (16)	13	Paylaşılmış UART/JTAG/ USB Port
6	Ledler (16)	14	Harici güç konektörü
7	Butonlar (5)	15	Açma-Kapama Anahtarı
8	FPGA programlanma tamamlandı LED'İ	16	Güç Seçim Jumper'ı

3.1.1. Pmod Konnektörleri

Pmod (Peripheral Module) konektörleri, Basys3 kartının dışındaki aygıtlarla iletişim kurmasını sağlamaktadır. Girişler ve çıkışlar bu konektörler üzerinde bulunmaktadır. Basys3 üzerinde, 4 adet, Şekil 3.3' de belirtildiği gibi pinlere sahip 2x6 Pmod konektör bulunmaktadır. [7]



Şekil 3.3 Pmod Konnektörü

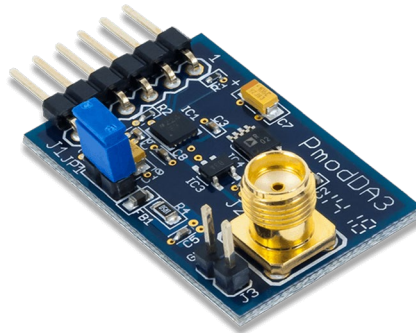


Şekil 3.4 Basys3 Pmod Konnektörleri

Şekil 3.4’de gösterilen JXADC portu, diğer Pmod portlarından farklıdır. Konfigürasyona göre, bu port diferansiyel analog sinyalleri, Artix-7 içindeki analog-dijital dönüştürücüye beslemek için kullanılabilmektedir. Bu porttaki pinler, analog giriş ya da dijital giriş-çıkış olarak ayarlanabilmektedir.

3.2. PMOD DA3

Pmod DA3, Digilent firması tarafından FPGA’larda ve Mikrodenetleyicilerde çevresel birimleri haberleştirmek için geliştirilmiş bir arabirim kartıdır. PmodDA3, yüksek çözünürlüklü ve düşük gürültülü analog çıkış için SMA konektörlü 16 bit dijital-analog dönüştürücüdür (DAC).[3]



Şekil 3.5 Pmod DA3

PmodDA3, SPI benzeri bir protokol aracılığıyla FPGA ile iletişim kurmaktadır. Bu arayüz, geleneksel SPI protokolünden farklıdır çünkü normalde veri almak için kullanılan MISO pini, artık yük DAC (LDAC) işlevi için kullanılmaktadır. Bu sayede, modül gelen 16 bit veriyle yüklendikten hemen sonra DAC'nin çıkışı güncellenebilmektedir.

Pmod'a veri göndermek için kullanıcıların Chip Select (CS) hattını mantık seviyesinde düşük voltaja indirmesi ve ardından SPI Mod 0'da 16 saat darbesi ve 16 bit veri göndermesi gerekmektedir. Bu, verinin en anlamlı bitinin (MSB) Seri Saat hattı (SCLK) mantık seviyesinde düşük voltaja getirildikten hemen sonra veri hattına yerleştirilmesi anlamına gelmektedir.

Tüm veriler toplanıp dahili seri giriş arabelleğinde tutulduktan sonra, CS hattını lojik seviyeye getirecek dahili seri giriş kaydedicisi için hazırlanan yüksek voltaj, LDAC'de iken kaydırma yazmacından tüm verileri dahili seri giriş kaydedicisine aktaracaktır. Yüksek voltaj durumu, LDAC'nin pininin önce düşük sonra yükseğe atılması, tüm bilgileri eş zamansız olarak DAC kaydına taşıyacak ve SMA'nın konektöründe uygun analog voltajın üretilmesine neden olacaktır. Alternatif olarak kullanıcılar, verileri kaydırma yazmacından DAC kaydına doğrudan iletmek için CS voltajını yükseltebilirken LDAC voltajını mantıksal seviyede düşük tutabilmektedir.

Tablo 2. Pmod DA3 Pinleri

Pin	Signal	Description
1	~CS	Chip Select
2	DIN	Data Input
3	~LDAC	LOAD DAC
4	SCLK	Serial Clock
5	GND	Power Supply Ground
6	VCC	Power Supply (3.3V/5V)

PmodDA3'e uygulanan herhangi bir harici güç, 2,7V ile 5,5V aralığında olmalıdır ancak Pmod'un 3,3V'ta çalıştırılması tavsiye edilmektedir.

Pmod DA3 Özellikleri

- Yüksek çözünürlüklü, 16-bit Dijital-Analog çevirici
- Düşük gürültülü analog çıkış
- SMA konektörü
- 2.5V referans voltajı
- Esnek tasarımlar için küçük PCB boyutu: 1.2" × 0.8" (3.0 cm × 2.0 cm)
- GPIO arayüzüne sahip 6-pin Pmod portu

BÖLÜM 4

4. PROJE KAPSAMINDA KULLANILAN UYGULAMALAR

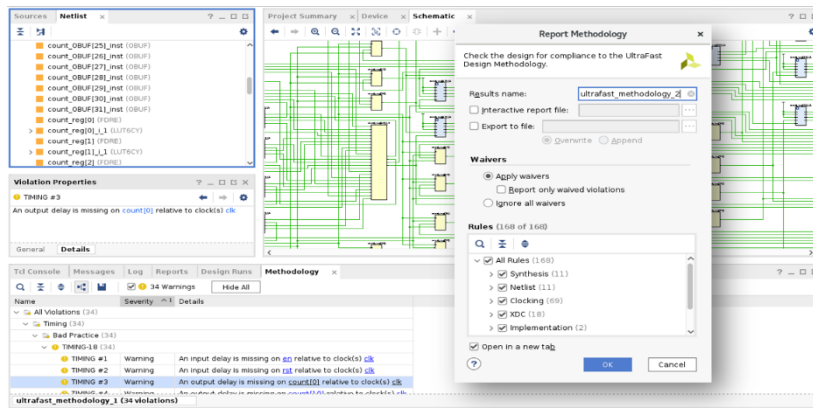
Bu projede uygulama olarak Xilinx Vivado, Matlab, Hterm ve Notepad++ kullanılmıştır. Bu bölümde proje kapsamında kullanılan uygulamalar ayrıntılı olarak anlatılmıştır.

4.1. Xilinx Vivado

Vivado, Xilinx tarafından geliştirilen bir yazılım aracı paketidir ve öncelikle Xilinx FPGA'leri (Alan Programlanabilir Kapı Dizileri) ve SoC'leri (Çip Üzerinde Sistem) tasarlamak ve programlamak için kullanılmaktadır. Karmaşık dijital sistemler oluşturmak için eksiksiz bir geliştirme ortamı sağlamaktadır. Prototip bir tasarım yapılmak istenirse, önce onun HDL'si yazılmalıdır. Xilinx Vivado, sentezlenmiş ağ listesindeki (dijital kapı seviyesi formatı) bu HDL'leri değiştirecek ve ayrıca FPGA için bit akışı üreterek bu kapıların çıkış değerlerini FPGA kartına yerleştirecektir. Vivado'nun temel kullanımı bu şekildedir. Ancak donanım yazılımı ortak simülasyonu, çevre birimlerinin panoya eklenmesi vb. gibi diğer farklı işlemler için de kullanılabilir. [1][2]

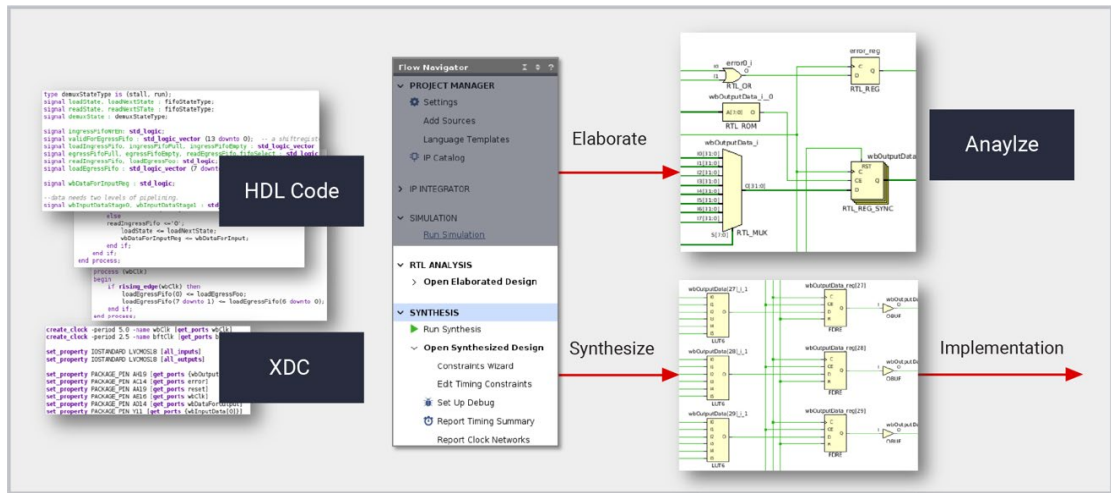
Vivado'nun temel özelliklerinden ve kullanımlarından bazıları şunlardır:

1. **Tasarım Metodolojisi:** Vivado, tasarımlara şemalar eklenmesine, Verilog ve VHDL gibi Donanım Tanımlama Dilleri ile üst düzey sentez gibi çeşitli yöntemleri kullanarak tasarım yapılmasına olanak sağlamaktadır. Tasarım metodolojisi Vivado ile kullanıldığında UltraFast metodolojisi uygun kısıtlamaların tanımlanmasına yardımcı olmakta, araçların doğru şekilde çalıştırılmasına ve sonuçların analiz edilmesine yardımcı olmakta ve genel üretkenliği artırmaktadır.



Şekil 4.1 Tasarım Metodolojisi

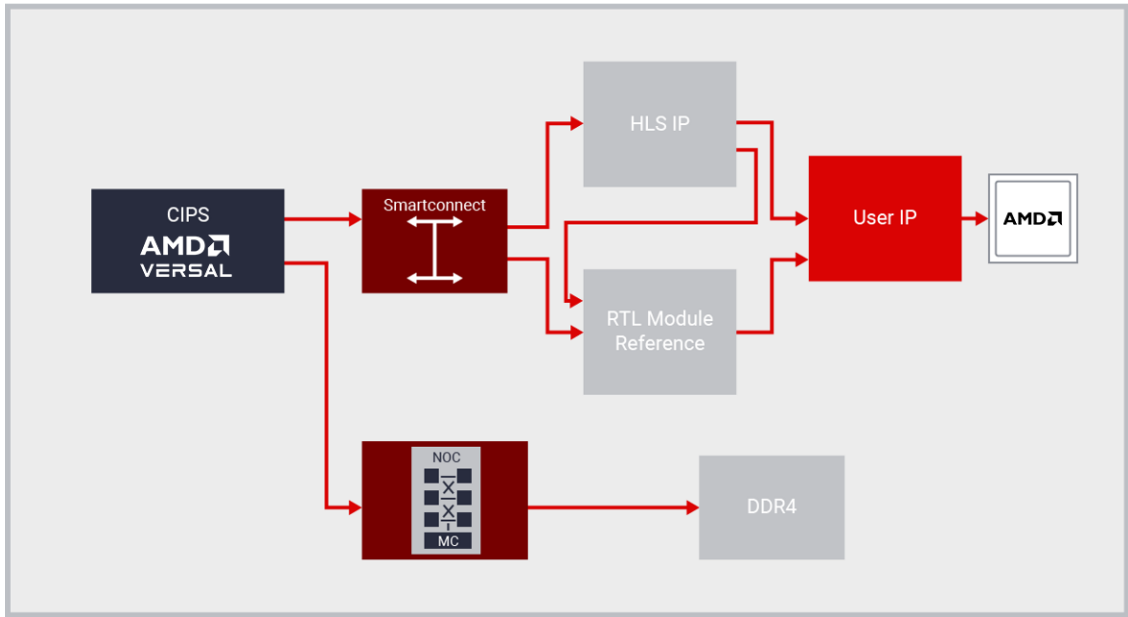
2. **Sentez:** Vivado mantık sentezi, IP ve özel tasarımlar üretmesine olanak tanıyan bir tasarım oluşturma aracıdır. Mantık sentezi, VHDL ve Verilog'da yazılan Kayıt Ak-tarım Seviyesi (RTL) tasarımlarını, aşağı yönde uygulama için kütüphane hücrele-rinin sentezlenmiş bir netliste dönüştürmektedir. Mantık sentezi, LUT RAM'ler, Blok RAM'ler, kaydırma yazmaçları, toplayıcı-çıkarıcılar ve DSP blokları dahil olmak üzere doğrudan özel silikon yapılarla eşlenen RTL açıklamalarından işlevler çıkarılabilmektedir. Sentez sonuçları, tasarım hedeflerine ulaşmak için nitelikler, araç seçenekleri ve Xilinx Tasarım Kısıtlamaları (XDC) kullanılarak yönlendiril-mektedir. Mantık sentezi, Vivado Projeleri ve Tcl komut dosyaları içinde çalışmak-ta ve Yüksek Düzey Sentez ve IP entegratörü de dahil olmak üzere RTL açıklama-ları üreten diğer üst düzey tasarım yöntemleri için sağlam bir temel sağlamaktadır.



Şekil 4.2 Sentez Örneği

3. **Uygulama:** Vivado, mantıksal tasarımı hedef cihazın fiziksel kaynaklarına eşleye-rek zamanlama, alan ve güç hususlarını optimize ederek yerleştirme ve yönlendir-me işlemini gerçekleştirmektedir.
4. **Programlama:** Vivado, istenen işlevselliği uygulamak için kullanıcıların yapılan-dırılmış FPGA veya SoC cihazını oluşturulan bit akışıyla programlamasına olanak tanımaktadır.

5. **Hata Ayıklama ve Analiz:** Vivado, mantık analizörleri, donanım hata ayıklayıcıları ve simülasyon araçları dahil olmak üzere tasarımda hata ayıklamak ve analiz etmek için araçlar sağlamaktadır.
6. **IP Entegrasyonu:** Vivado, Xilinx veya üçüncü taraf satıcılar tarafından sağlanan fikri mülkiyet (IP) çekirdeklerinin tasarıma entegrasyonunu desteklemektedir.



Şekil 4.3 IP Entegrasyonu

Genel olarak Vivado, FPGA ve SoC tasarımları için geliştirme sürecini kolaylaştıran, tasarım, sentez, uygulama, programlama, hata ayıklama gibi özellikleri sunan kapsamlı bir araçtır.

4.2. Matlab

MATLAB ("MATrix LABoratory"nin kısaltması), MathWorks firması tarafından geliştirilen sayısal hesaplama dilidir. Aynı zamanda pozitif bilim ve mühendislik hesaplamaları için kullanılan bir bilgisayar programıdır. MATLAB (Matrix Laboratory) isminden de anlaşılacağı üzere matris tabanlı çalışma sistemine sahiptir. MATLAB ile yapılabilen işlemler şu şekildedir:[5]

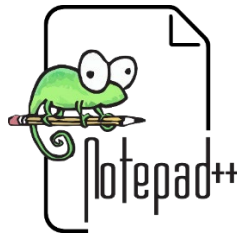
- Simülasyon ve Modelleme,
- Lineer cebir, istatistik, Fourier analizi, filtreleme, optimizasyon, sayısal integrasyon vb. konularda matematik fonksiyonlar,
- Algoritma geliştirme,
- 2D ve 3D grafiklerin çizimi,
- Uygulama Geliştirme (Kullanıcı Arayüzü Oluşturma)



4.3. Notepad++

Notepad++, Windows işletim sistemi içerisine gömülü olarak gelen Notepad [1] yazılımının yerine kullanılmak üzere C++ ile saf Win32 API ve STL ile geliştirilmiş GPL ile dağıtılan açık kaynak kodlu bir kaynak kod düzenleyicisidir. Scintilla düzenleyicisi temel alınarak geliştirilen Notepad++, birçok yazılım diline ait kodu görüntüleyip uygun olarak renklendirebilir. Türkçe dâhil birçok konuşma diline destek veren bir arabirimi bulunmaktadır.[4]

Desteklenen Diller: C, C++, Java, C#, XML, HTML, PHP, CSS, makefile, ASCII, art (.nfo), doxygen, ini dosyası, batch dosyası, JavaScript, ASP, VB/VBS, SQL, Objective-C, Pascal, Perl, Python, Lua, TeX, TCL, Assembler, Ruby, Lisp, Scheme, Diff, Smalltalk, Postscript, VHDL, Ada, Caml, AutoIt, KiXtart, Matlab, Verilog, Haskell, InnoSetup, CMake, YAML gibi programlama dilleri desteklenmektedir.



4.4. Hterm

Hterm, Windows ve Linux üzerinde çalışan seri iletişim terminal programıdır. Seri port (USB) üzerinden haberleşme yapmaya olanak tanır. Hterm'in başlıca özellikleri aşağıdaki şekildedir:

- 1) Veri Gönderme ve Veri Alma: Kullanıcılar, Hterm aracılığıyla seri port üzerinden cihazlarına veri gönderebilir ve cihazlarından veri alabilirler. Bu özellik, özellikle veri iletiminin ve alımının önemli olduğu uygulamalarda büyük kolaylık sağlar.
- 2) Seri Port İletişimi: Hterm, RS232 ve diğer seri port standartlarını destekleyerek, seri port ile çalışan cihazlarla güvenilir bir iletişim kurulmasını sağlar. Bu, mikrodenetleyiciler ve diğer gömülü sistemlerle yapılan projelerde kritik bir öneme sahiptir.
- 3) Bağlantı Parametrelerinin Ayarlanması: Hterm, baud hızı, veri bitleri ve parite gibi çeşitli seri iletişim parametrelerinin kolayca ayarlanmasına imkân tanır. Bu sayede, farklı cihazlarla uyumlu bir bağlantı sağlanabilir.
- 4) Komut Gönderme: Kullanıcılar, Hterm aracılığıyla cihazlarına belirli komutlar gönderebilir ve bu komutların sonuçlarını anında gözlemleyebilirler. Bu özellik, cihazların konfigürasyonu ve test edilmesi sırasında oldukça kullanışlıdır.

BÖLÜM 5

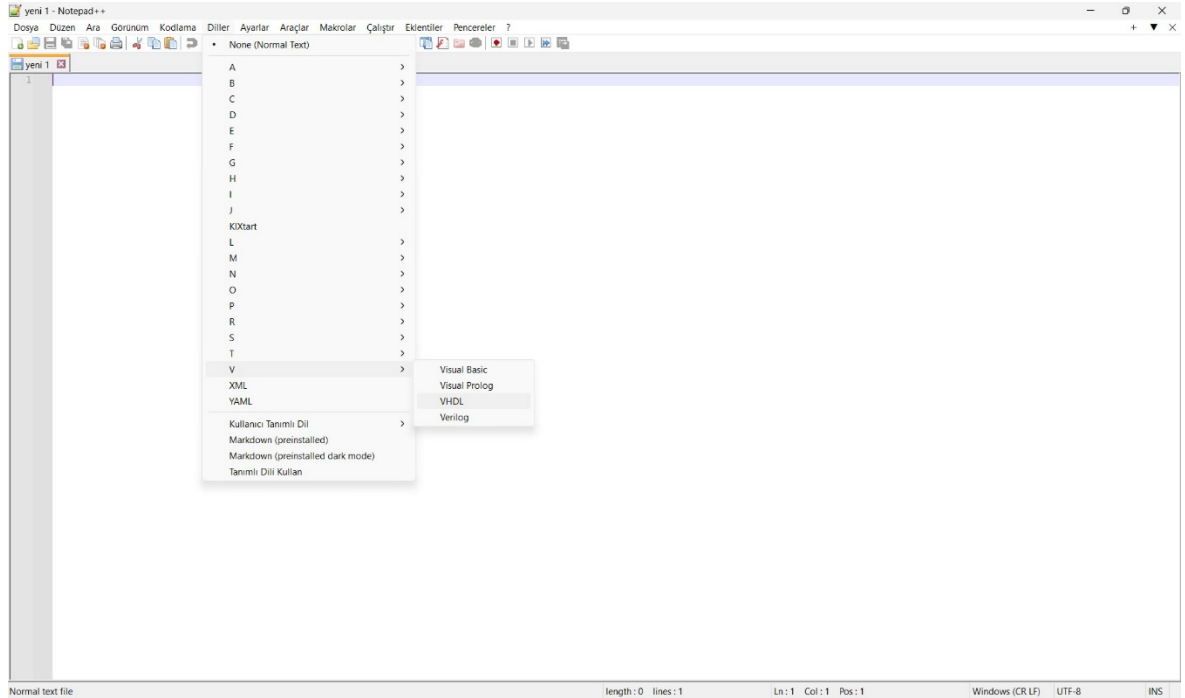
5. PROJE KAPSAMINDA YAPILAN İŞLEMLER

Bu bölümde proje kapsamında yapılan işlemler ve kullanılan protokoller, yazılımsal ve donanımsal tasarım ele alınmaktadır.

Proje kapsamında gerekli yazılımları oluşturmak için Notepad++, projenin tasarımı ve programlanması amacıyla Xilinx Vivado, sinyal katsayılarını oluşturmak için Matlab, oluşturulan katsayıları FPGA'ye göndermek için Hterm uygulamaları kullanılmıştır.

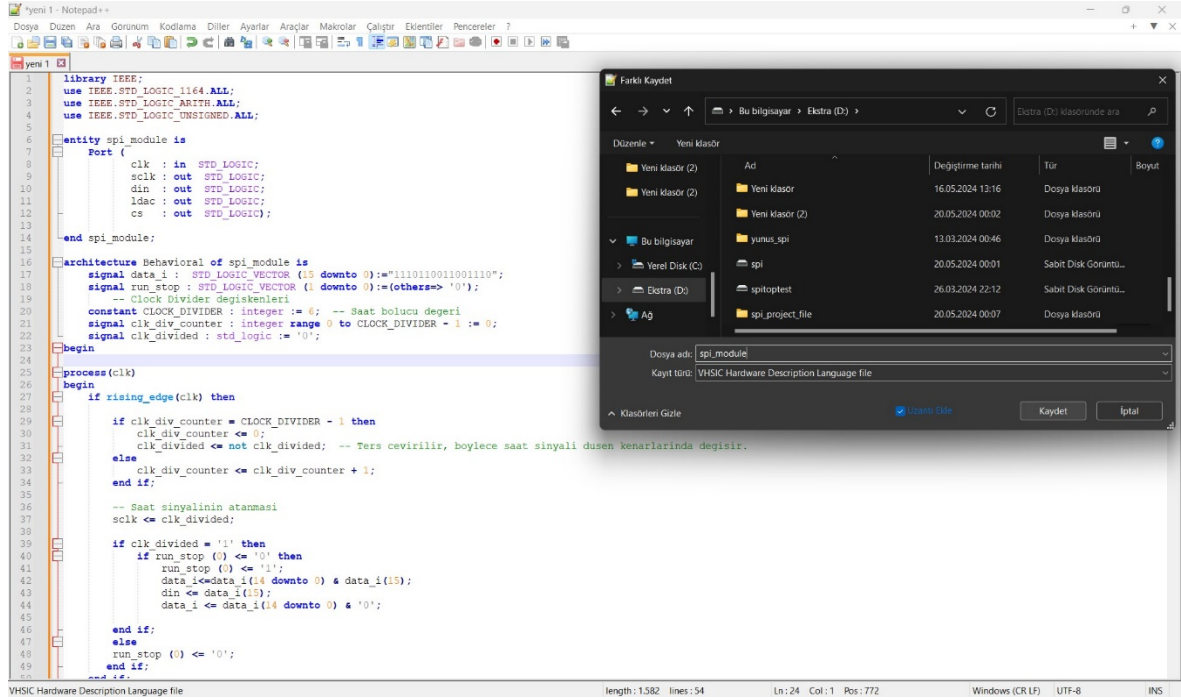
5.1. Notepad++ ile Yazılım Oluşturma

Proje kapsamında VHDL dili kullanılmıştır. Notepad++ uygulamasında VHDL yazılımı oluşturmak amacıyla Şekil 5.1'deki gibi menü bar üzerinde bulunan diller seçeneğinden VHDL dili seçildi.



Şekil 5.1 Notepad++ Uygulama Ekranı

Gerekli yazılımın hazırlanmasından sonra Şekil 5.2’deki gibi Kaydet butonuna tıklanarak istenilen konuma kaydedildi.



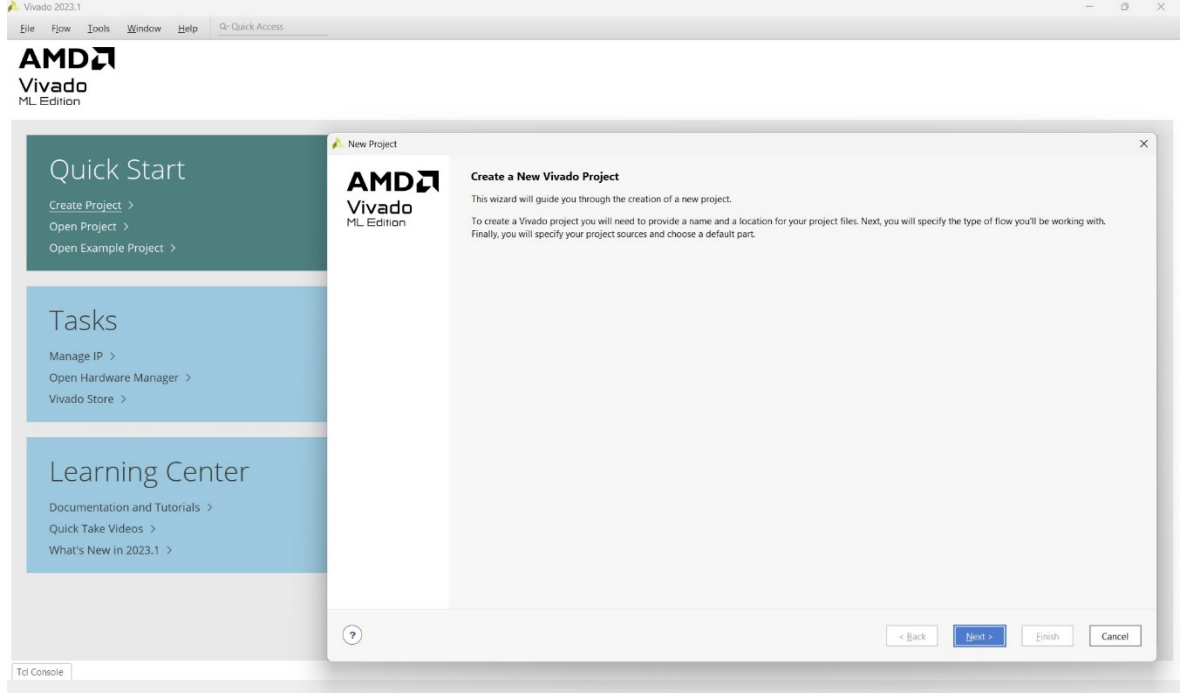
Şekil 5.2 Notepad++ Oluşturulan Yazılımın Kaydedilmesi

5.2. Vivado Uygulaması

Bu bölümde Vivado uygulaması ile proje oluşturma, simülasyon ve FPGA programlama işlemleri detaylı olarak ele alınmıştır.

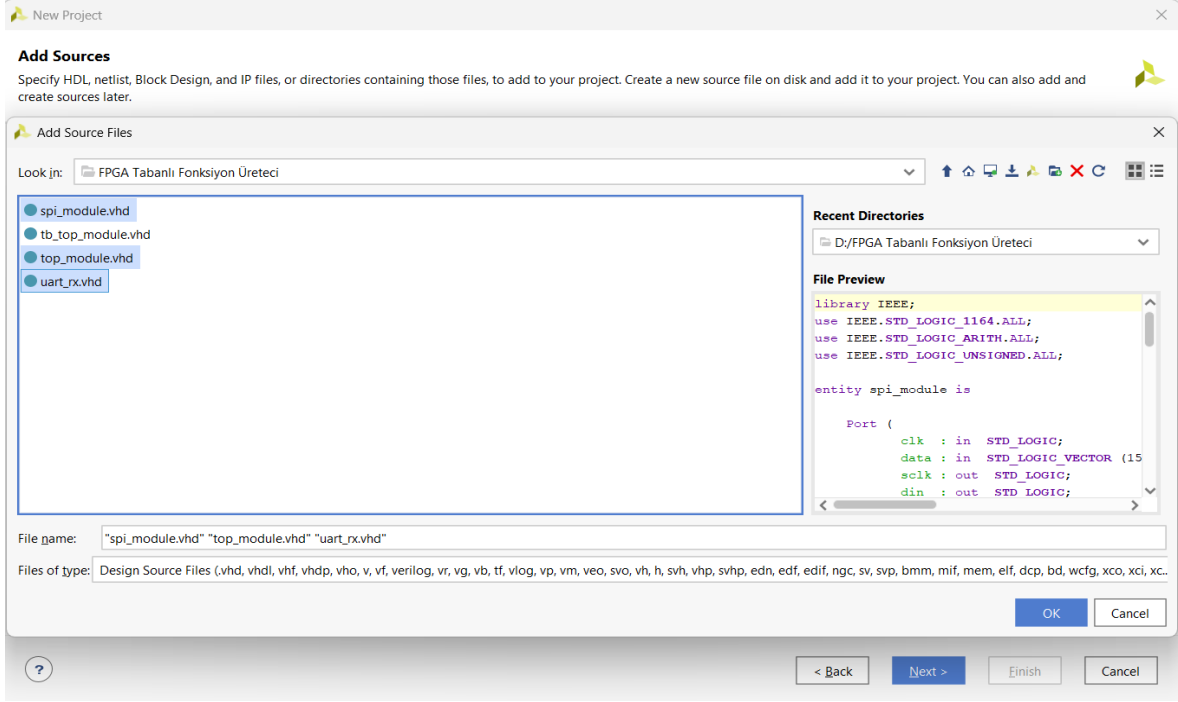
5.2.1. Vivado Proje Oluşturma

Proje kapsamında Vivado 2023.1 kullanılmıştır. Şekil 5.3’de görülen ekran üzerinde Create New Project ifadesi seçilerek yeni bir proje oluşturma penceresi açılmıştır.



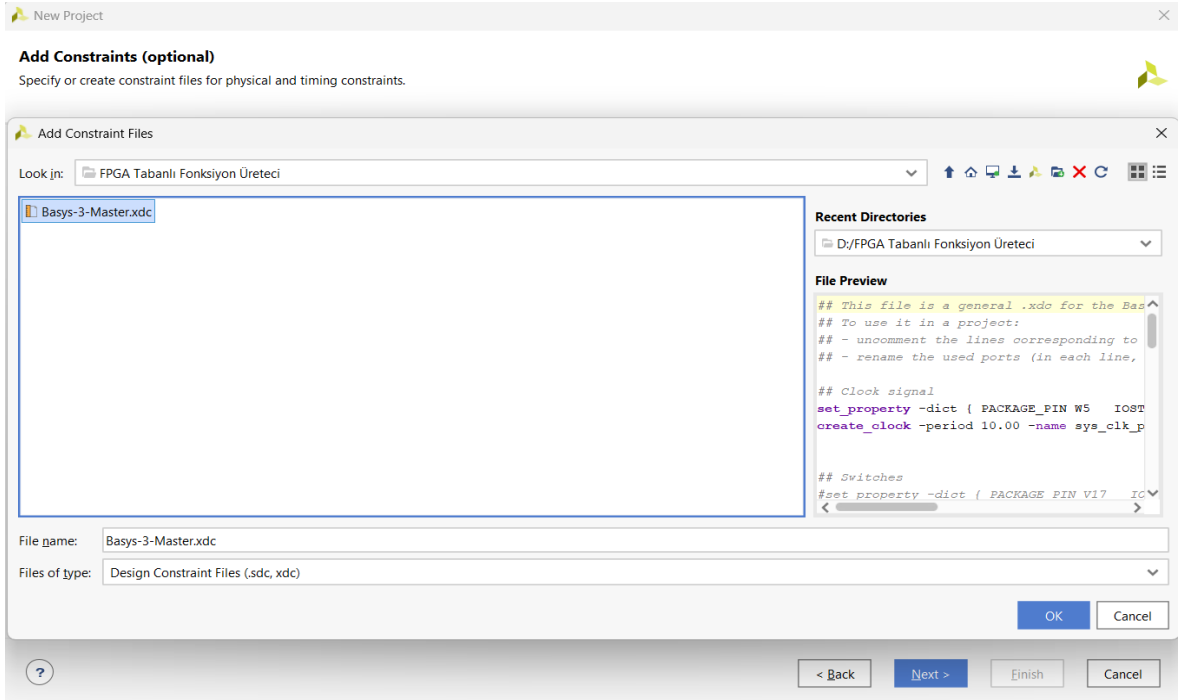
Şekil 5.3 Vivado Proje Oluşturma Ekranı

Açılan pencerede Next butonuna tıklanarak bir sonraki aşamaya geçilmiştir. Bu bölümde oluşturulan proje için bir isim ve projenin bulunması istenilen konum seçilmiştir. Next butonuna tıklanarak bir sonraki ekrana geçilmiştir. Açılan ekranda Proje Tipi olarak RTL Project ifadesi otomatik seçili bulunmaktadır ve herhangi bir değişiklik yapılmadan Next butonu ile bir sonraki ekrana geçiş yapılmıştır. Şekil 5.4’de görülen Add Sources ekranında Add Files butonuna tıklanarak daha öncesinde Notepad++ ile oluşturulan spi_module, uart_rx ve top_module VHDL yazılımları projeye eklenmiştir. Target Language ve Simulator Language seçenekleri VHDL olarak seçilmiştir ve Next butonu bile bir sonraki ekrana geçilmiştir.



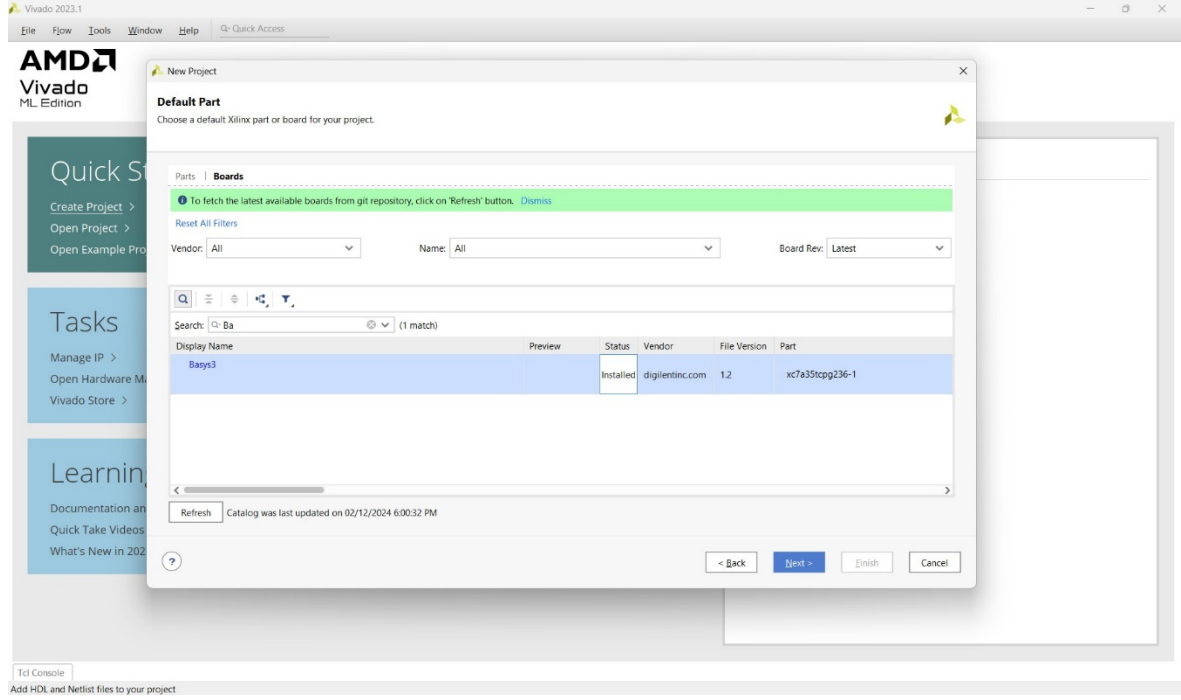
Şekil 5.4 Vivado Proje Kaynak Dosyası Ekleme

Bir sonraki ekran olan Add Constraints ekranında opsiyonel olarak constraint dosyası projeye eklenebilmektedir. Şekil 5.5’de görüldüğü gibi Add files butonu ile Basys3’e ait olan constraint dosyası projeye eklenmiştir.



Şekil 5.5 Vivado FPGA Constraint Dosyası Ekleme

Bir sonraki ekrana geçildiğinde projenin geliştirilecek olduğu FPGA Board’u (Basys3) Şekil 5.6’da görüldüğü gibi seçilmiştir. Basys3 seçeneğinin bulunmadığı durumlarda kütüphaneye eklenebilmektedir. Next butonu ile board ekranı geçilir ve sonraki ekranda yeni oluşturulan projenin özet bilgileri bulunmaktadır. Bu ekranda da Finish butonuna tıklanarak proje oluşturulmuştur.



Şekil 5.6 Vivado FPGA Bord Seçimi

5.2.2. Vivado’da Simülasyon ile Tasarımın Doğrulanması

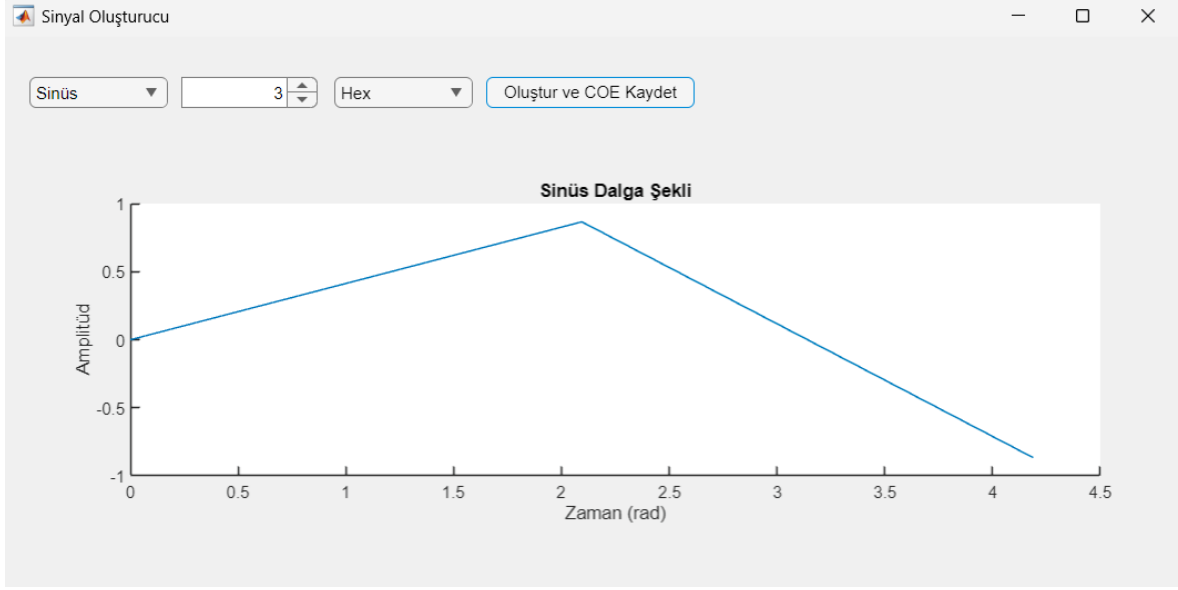
Vivado’da oluşturulmuş olan proje ekranında Sources bölümü içinde bulunan Simulation Sources seçeneğine sağ tıklanarak Şekil 5.7’de belirtildiği gibi Add Sources menüsü üzerinden simülasyon kaynağı ekle veya oluştur seçeneğinin seçilmesiyle simülasyon dosyası eklenmiştir. Projenin doğrulanması amacıyla Vivado üzerinde Simulation menüsü altından Run Simulation butonuna tıklanmıştır. Daha sonra Run Behavioral Simulation butonuna tıklanarak Simülasyon işlemi başlatılmıştır. Şekil 5.8’de projenin Simülasyon ortamında çalışma şekli görülmektedir. Projenin simülasyon ortamında gözlemlenebilmesi için tasarlanan yazılım EK 6. TestBench Yazılımında belirtilmiştir.

5.2.3. Vivado ile Oluşturulan Projenin FPGA’ye Yüklmesi

Vivado ile oluşturulan projeye ait kaynak dosyalarının ve FPGA Bordu’na ait constraint dosyasının eksiksiz olarak yüklenmesi gerekmektedir. FPGA programlanması için Simülasyon dosyası gerekmemektedir. Design Sources ve Constraints dosyalarının yüklenmesi yeterlidir. Kaynak dosyalarının yüklenmesinden sonra sırasıyla Run Synthesis, Run Implementation ve Generate Bitstream işlemleri yapılmalıdır. Daha sonra FPGA, USB aracılığıyla bilgisayara bağlanmalıdır. Bağlantı işleminin tamamlanmasıyla Vivado üzerinden Open Hardware Manager menüsü altında Open Target, Auto Connect seçeneği seçilmelidir. Açılan pencereden kullanılan FPGA bordu seçilerek program device seçeneği ile FPGA programlanmıştır.

5.3. Matlab Uygulaması

Matlab kullanılarak sinyal katsayılarını oluşturabilen ve oluşturulan katsayıları coe formatında dosyaya kaydeden uygulama geliştirildi. Sinyal tiplerini seçebilme, örnekleme sayısını belirleme, dec veya hex olarak veri türünü seçebilme özelliklerine sahip bir uygulama arayüzü tasarlandı. Ayrıca oluşturulan sinyali görebileceğimiz bir grafik arayüze eklendi. Şekil 5.9’da görülen uygulama üzerinden sinyal tipleri (sinüs, üçgen, kare, testere) seçilir, örnekleme sayısı 1 ile 10000 arasında belirlenir, veri formatı (hex, dec) seçilir ve sinyal oluşturma işlemi için "Oluştur ve COE Kaydet" butonuna tıklanmıştır. “generateSignal” fonksiyonu, kullanıcının seçtiği sinyal türüne göre örnekleme noktaları ve ilgili sinyal verileri hesaplanmaktadır. Oluşturulan sinyal verileri “writeCOEFile” fonksiyonu aracılığıyla COE dosyası formatında kaydedilmektedir. FPGA gibi donanım tasarım süreçlerinde kullanılmak üzere verilerin belleğe yüklenmesi için gerekli olan formatta kaydedilmektedir. EK 4. Matlab Yazılımında Matlab uygulamasına ait yazılım belirtilmiştir.



Şekil 5.9 Matlab Uygulama Ekranı

5.4. Hterm Uygulaması

FPGA'e Uart Haberleşme Protokolü aracılığıyla veri gönderimini sağlamak amacıyla Hterm uygulaması kullanılmıştır. Vivado üzerinden FPGA kartının programlanmasından sonra Hterm uygulaması açılmaktadır. Uygulama ekranından haberleşmenin yapılacağı port olarak "COM4" seçilmektedir. Baud ifadesinin bulunduğu seçeneklerden baudrate oranı seçilmektedir. Baudrate oranı yazılımda 9600 olarak belirlendiğinden Hterm üzerinden de 9600 değeri seçilmelidir. Daha sonra uygulama ekranında bulunan Ascii, Hex, Dec, Bin ifadelerinden gönderilecek veri tipine göre seçim yapılmaktadır. Bu projede Hex (Hexadecimal) cinsinden veri gönderimi sağlanmıştır. Son olarak da "Connect" butonu ile FPGA bordu ile bağlantı sağlanmıştır. FPGA'e gönderilecek olan veri Input kontrol bölümünde bulunan box'a yazılır ve "Autosend" butonuna tıklanarak FPGA'e gönderilir. Yazılımda verilerin kaydedildiği array'in kapasitesi 100 veri alacak şekilde ayarlanmıştır. Dolayısıyla maksimum 100 adet 16 bitlik veri gönderilmelidir. Gönderilen veriler "Transmitted Data" bölümünde görülmektedir.

5.5. Haberleşme Protokolü

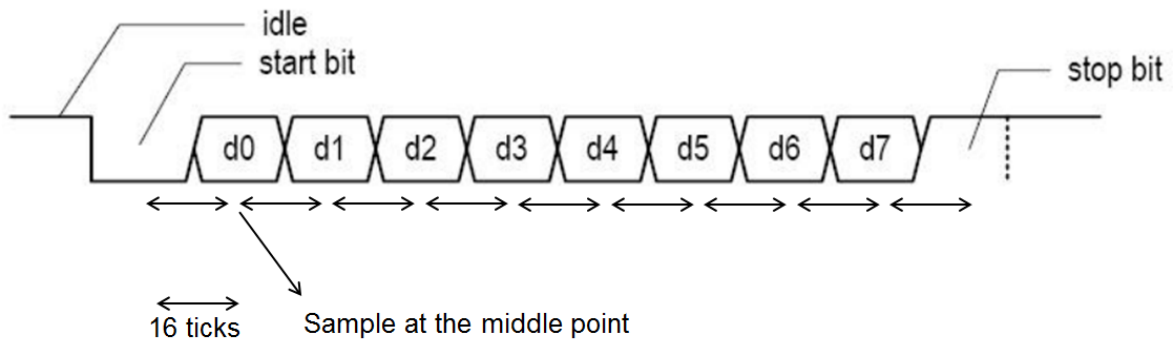
Bilgisayardan FPGA'ye veri gönderilebilmesi amacıyla Bilgisayar ile FPGA arasında haberleşmeyi sağlamak için UART protokolü kullanılmasına karar verilmiştir. FPGA'ye gönderilen verilerin DAC (PmodDA3) 'a gönderilmesi amacıyla SPI Protokolünün kullanılması kararlaştırılmıştır.

5.5.1. UART (Universal asynchronous receiver/transmitter) Protokolü

UART (Universal Asynchronous Receiver Transmitter – Evrensel Eş Zamanlı Olmayan Alıcı/Verici) paralel ve seri formlar arasında veri çevrim işlemini gerçekleştiren bilgisayar donanım parçasından biridir. RS-232, RS-485 gibi yaygın iletişim standartlarıyla birlikte kullanılmaktadırlar.

UART Haberleşmesi ile FPGA' e veri gönderme işlemi sağlanmıştır. Uart_Rx modülü sayesinde bilgisayardan gönderilen veri FPGA' e ulaşmaktadır.

UART modülü, Şekil 5.10'da belirtildiği gibi idle durumunda logic 1 seviyesindedir. Logic 0 durumuna geldiğinde start bit ile başlamaktadır. Start biti T periyot kadar 0'da kalmaktadır. Sonrasında öncelikle LSB (0.bit) iletilecek şekilde 8 bitlik veri gönderilmektedir. 8 bit tamamlandıktan sonra 1 veya veri kaçırmanın önlenmesi amacıyla 2 bitlik stop bit'i gönderilerek 1 byte veri gönderimi tamamlanmaktadır.[12]

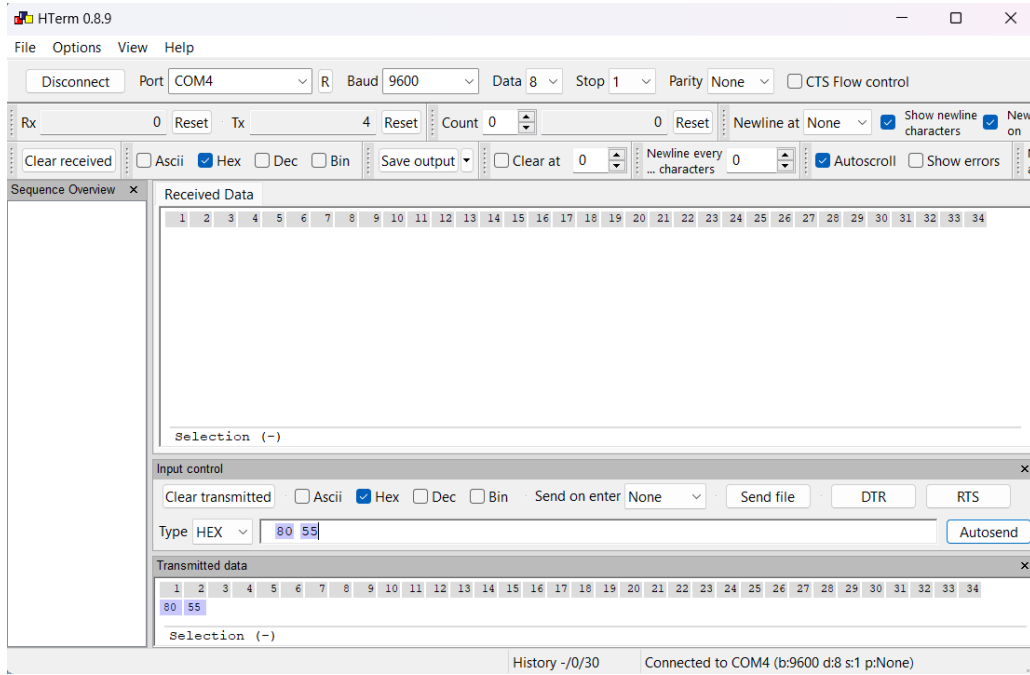


Şekil 5.10 UART Receiver

UART yazılımının doğrulanması amacıyla Testbench kodu hazırlanarak Vivado üzerinde Simülasyon ortamında gözlemlenmiştir. Daha sonra donanım üzerinde gerekli testler yapılmıştır.

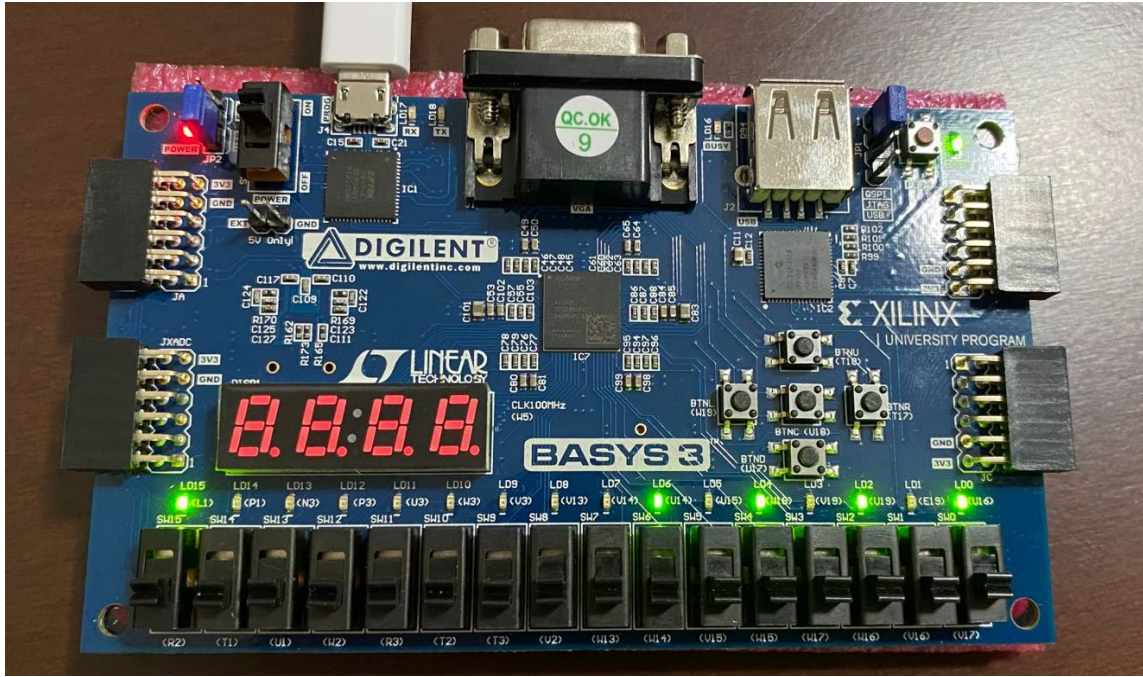
Şekil 5.11’de belirtildiği gibi UART yazılımı aracılığıyla Hterm uygulaması üzerinden 8 bit olacak şekilde veri gönderilmiştir. PmodDA3 (DAC)’ e gönderilecek verilerin 16 bit olması sebebiyle Matlab’daki uygulamada sinyal katsayıları 16 bit olarak oluşturulmuştur. Oluşturulan 16 bitlik katsayılar Hterm’e yazıldığında otomatik olarak 8 bit şeklinde ayrılmaktadır. Bilgisayardan UART aracılığıyla gönderilen 8 bitlik veriler FPGA’ye ulaştıktan sonra ilk giden 8 bitlik veri 15-8. Bitlere, ikinci 8 bitlik veri 7-0. Bitlere yazılmaktadır. Örneğin; “23 12 55 00” şeklinde veri gönderildiğinde, verilerin depolandığı data_array’e ilk önce 2312 daha sonra 5500 verileri 16 bit olacak şekilde yazılmaktadır.

UART modülü ile alakalı yazılım EK 1. UART_RX de belirtilmiştir.



Şekil 5.11 Uart Haberleşmesi ile Hterm Üzerinden Veri Gönderimi

Gönderilen bu verilerin FPGA'ye ulaştığını doğrulamak amacıyla FPGA üzerinde bulunan ledler kullanılmıştır. Şekil 5.12'de görüldüğü gibi ilk 8 bitlik veri ledlerin 15 ile 8. Bitlerini yakmaktadır. Gönderilen ikinci 8 bitlik veri de 7 ile 0. Bitlerini yakmaktadır. Böylece FPGA' gönderilmiş olan 8 bitlik iki veri 16 bitlik veri olarak ledlerde gözlemlenmiştir.



Şekil 5.12 Gönderilen Verilerin FPGA üzerindeki Ledlerde Gösterimi

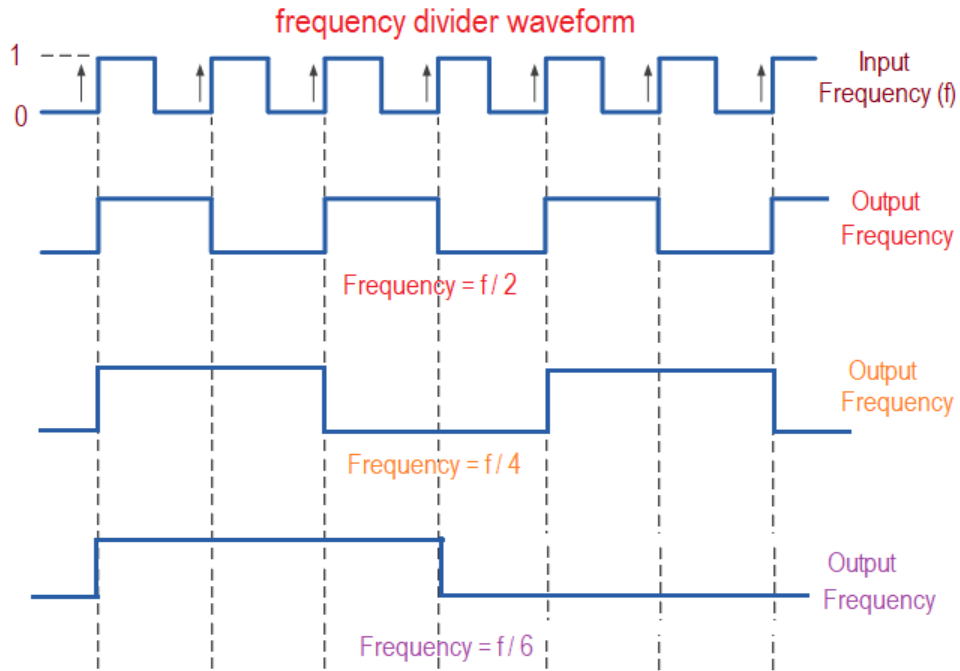
Projenin son durumunda gönderilen verilerin hızlı bir şekilde iletilmesi ve sürekli olarak döndürülmesinden kaynaklı olarak ledlerde verilerin gözlemlenmesi mümkün olmamıştır. Bu sebeple gönderilen verileri okuyan sayaç değeri ledlere atanmıştır. Aynı zamanda bu sayaç değeri butonlar ile kontrol edilebilmektedir. Örneğin; 100 veri gönderildiğinde, sayaç değeri 1 olarak ayarlanırsa 100 verinin tamamı okunmaktadır ve ledlerde 1'e karşılık gelen bit yanmaktadır.

5.5.2. SPI (Serial Peripheral Interface) Protokolü

SPI (Serial Peripheral Interface) kısa mesafeli işlemlerde, özellikle gömülü sistemlerde kullanılmaktadır. Master-Slave yapısı ile çalışmaktadır. Master için FPGA örnek verilebilmektedir. SPI, tam çift yönlü bağlantıya sahiptir, bu da verilerin aynı anda gönderilip

alınması anlamına gelmektedir. Yani bir master, bir slave'e veri gönderebilmekte ve bir slave, aynı anda master'a veri gönderebilmektedir.

SPI Haberleşmesi için gerekli yazılım aşamalı olarak oluşturulmuştur. İlk aşamada Pmod-DA3'ün Datasheet' inden yararlanılarak gerekli yazılım tasarlanmıştır. Bu aşamada PmodDA3 için gerekli olan zaman diyagramı tasarımı yapılmamıştır. Öncelikli amaç SPI yazılımı ile Pmod'un "DIN (serial data input)" portuna iletilen verinin ulaşp ulaşmadığının kontrolünü sağlamaktır. Ayrıca bu aşamada clock divider (saat bölücü) işlemi uygulanarak Basys3'nin dahili clock frekansının kontrolü sağlandı. Şekil 5.13' de clock divider (saat bölücü) uygulamasına ait bir örnek bulunmaktadır. Basys3'nin clock frekansı 100 MHz iken Sclk frekansı 50 MHz'dir. Clock divider değeri 5 olarak ayarlandığında clock frekansı 20 MHz iken Sclk frekansı 10 MHz olarak gözlemlenmiştir. Tüm bu kontroller FPGA'in Pmod portunda yer alan pinlerden alınarak osiloskop üzerinde gözlemlenerek sağlanmıştır.



Şekil 5.13 Clock Divider

SPI Haberleşmesi için yapılan çalışmaların son aşamasında önceki aşamada kullanılan yazılıma ek olarak case yapısı, Pmod DA3 Datasheet'inde yer alan zamanlama özellikleri ile Şekil 5.14'de belirtilen zamanlama diyagramı göz önünde bulundurularak eklenmiştir.

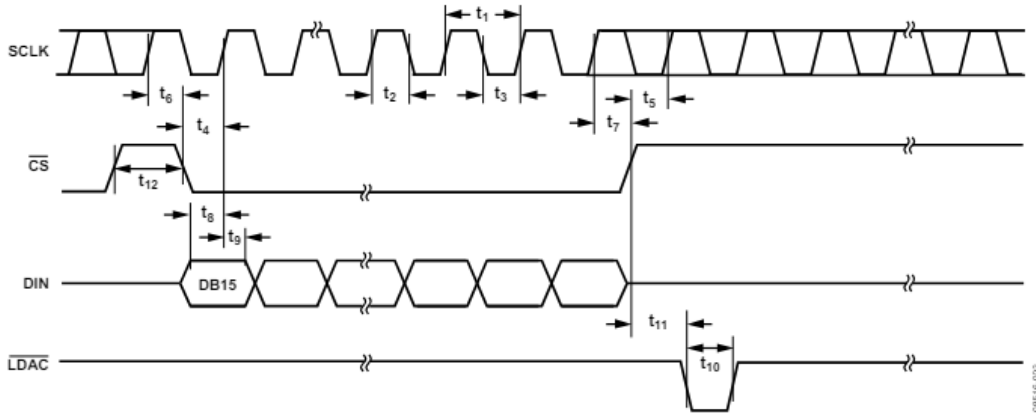
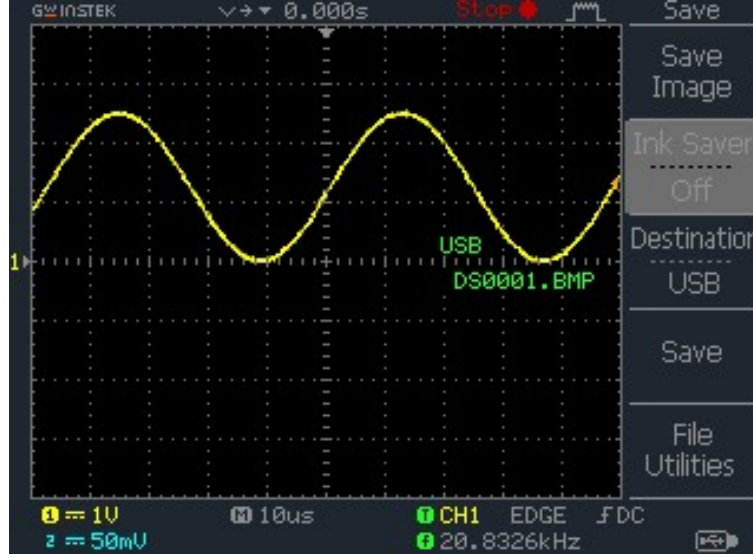


Figure 3. Timing Diagram

Şekil 5.14 PmodDA3 Zaman Diyagramı

Yazılım Vivado uygulamasında oluşturulan projeye yüklenmiştir. FPGA ile PmodDA3 bağlantısı yapılmıştır. FPGA programlanarak 16 bitlik örnek bir veri gönderilmiştir. PmodDA3'ün çıkışı osiloskopa bağlanmıştır ve gönderilen veri osiloskop ekranında gözlemlenmiştir. SPI Haberleşmesinin sorunsuz çalıştığını doğrulamak amacıyla birden fazla 16 bitlik veri gönderimi yapılarak test edilmiştir. Yapılan testlerde birden fazla verinin gönderilebildiği ve output olarak alındığı gözlemlenmiştir. Birden fazla verinin kolaylıkla gönderilebilmesi amacıyla yazılıma Array mantığı eklenmiştir. Bir sinüs sinyali oluşturmak için Array'e 100 adet 16 bitlik sinüs sinyalinin katsayı verileri tanımlanmıştır. PmodDA3'ten alınan çıktı osiloskopta görüntülendiğinde sinüs sinyali Şekil 5.15'de ki gibi düzgün bir biçimde elde edilmiştir. Ancak verilerin seri olarak gönderilmesinden kaynaklı olarak 50 MHz sclk frekansına sahip projede, yazılımda bulunan index boyutunun 23 olmasından kaynaklı olarak elde edilebilecek sinyalin maksimum frekans değeri 50/23 olarak hesaplandığında yaklaşık 2,174 MHz elde edilmektedir. Ayrıca bu frekans değerinin veri sayısının artması ile azaldığı görülmüştür. Örneğin 100 adet veri için elde edilen sinyalin frekans değeri yaklaşık 2 KHz'dir. Gönderilen veri sayısı 1 adet olduğunda elde edilen sinyalin frekans değeri yaklaşık 2 MHz'dir. Array mantığının ve PmodDA3 için gerekli olan zamanlama diyagramına uygun case yapısının bulunduğu gerekli yazılım EK 5. SPI ile Veri Gönderme Modülünde belirtilmiştir.[6]



Şekil 5.15 Sinüs Dalga Formu

Projenin son durumunda gönderilecek veri Bilgisayar üzerinden UART aracılığıyla aktarılacağından SPI modülünde bulunan array mantığı kaldırılmıştır. SPI, UART'dan gelen veriyi alarak işleyecektir. SPI modülü ile ilgili yazılım EK 2. SPI Modülünde belirtilmiştir.

5.6. Üst Modül Oluşturulması

UART ve SPI modülleri kullanılarak bir üst modülde veri gönderme işlemleri, gönderilen verinin kontrolü gibi işlevler tasarlanmıştır. Bilgisayardan UART aracılığıyla gönderilen 8 bitlik verilerin FPGA'ye ulaştıktan sonra ilk giden 8 bitlik verinin 15-8. Bitlere, ikinci 8 bitlik verinin 7-0. Bitlere yazma işlemi üst modülde gerçekleştirilmektedir. Tüm veriler bu şekilde yazıldıktan sonra SPI ile PmodDA3'e gönderilmektedir.

Üst modülde bulunan send_index_value sinyali ile array'e yazılan verilerden kaç tanesinin okunacağı belirlenebilmektedir. Kontrolü FPGA üzerinde bulunan butonlar aracılığıyla sağlanmaktadır. BTNU ile artırılabilen, BTND ile azaltılabilmektedir. Aynı zaman da maksimum üst sınırı 50 olarak ayarlanmıştır. Böylece array de bulunan verileri sürekli olarak okuduğunda herhangi bir karışıklığa sebep olmamaktadır. Send_index_value değeri 50 olduğunda 100 adet veriden sadece 2 veri okunmaktadır. Böylece frekans 1MHz olarak elde edilebilmektedir. Üst modül ile alakalı yazılım EK 3. Üst Modülde belirtilmiştir.

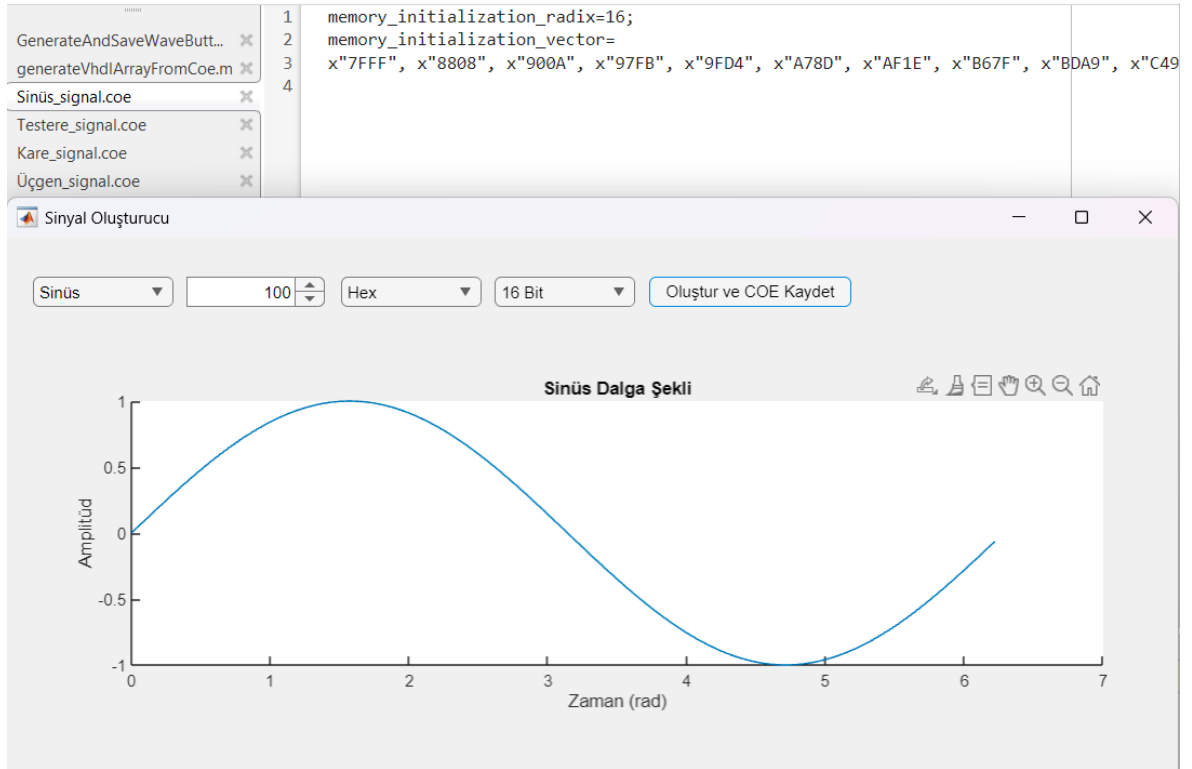
BÖLÜM 6

6. PROJENİN KULLANIMI VE SİNYAL OLUŞTURULMASI

Bu bölümde projenin kullanımı ve farklı sinyal türlerinin elde edilmesi detaylı olarak ele alınmaktadır.

6.1. Matlab'da İstenilen Sinyal Katsayılarının Üretilmesi

Matlab da istenilen formda sinyal katsayılarının üretilmesi için Şekil 6.1'de belirtildiği gibi sinyal tipi seçilerek, katsayı sayısı array boyutu 100 olduğundan 100 adet olarak belirlenmelidir. Ardından veri tipi hex ve 16 bit olarak seçilerek katsayılar oluşturulmalıdır. Oluşturulan katsayılar Matlab da bulunan sinyal tipine göre coe formatında farklı dosyalara kaydedilmektedir. Seçilen sinyal tipine göre dosya açılarak verilere erişim sağlanabilmektedir.



Şekil 6.1 Matlab'da Katsayı Oluşturulması

6.2. FPGA (Basys3) Kartının Programlanması

Vivado’da proje açılarak sentezleme, implementasyon ve bitstream oluşturma işlemleri yapıldıktan sonra FPGA’in bilgisayara Micro-USB kablo ile bağlantısı yapılmaktadır. Vivado üzerinden Open Target ve Auto Connect seçenekleri ile FPGA programlanmaktadır.

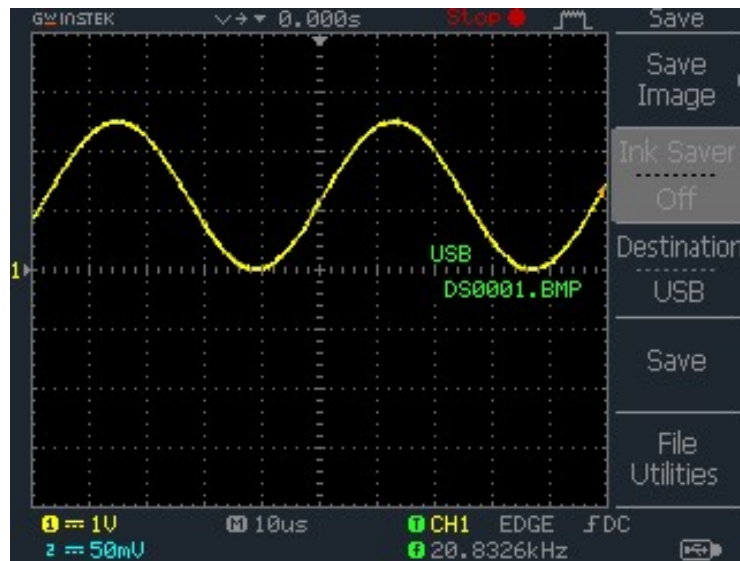
6.3. Hterm ile Veri Aktarımı

FPGA programlandıktan sonra Hterm uygulaması üzerinden Matlab’da oluşturulan veriler 5.4’ de anlatıldığı şekilde FPGA’e gönderilmektedir.

6.4. Farklı Sinyal Formlarının Üretilmesi

6.4.1. Sinüs Dalga Formu

Matlab’da verilerin oluşturulması esnasında Sinüs sinyal tipi seçilerek oluşturulmaktadır. Oluşturulan 100 adet veri FPGA’e aktarılarak sinüs sinyali oluşturulmaktadır. Butonlar ile kontrolü sağlanan ve aynı zamanda FPGA üzerindeki ledler aracılığıyla takibi yapılabilen “send_index_value” sinyalinin değeri artırılarak 1 olarak ayarlandığında 100 adet verinin tamamı çıkışa aktarılmaktadır. 100 adet veri ile Şekil 6.2’de belirtildiği gibi sinüs sinyali oluşmaktadır. Frekans değeri 20,8326 KHz olarak elde edilmiştir.



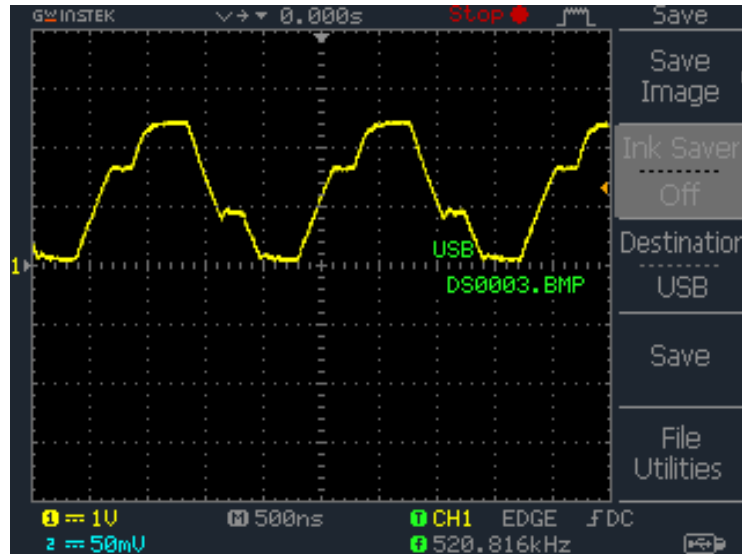
Şekil 6.2 100 Adet Veri ile Sinüs Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 10 değerine ulaştığında FPGA üzerinde binary cinsinden 10’a karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 10 veri aktarılmaktadır. 10 adet veri ile Şekil 6.3’de belirtildiği gibi sinüs sinyali oluşmaktadır. Frekans değeri 208,326 KHz olarak elde edilmektedir.



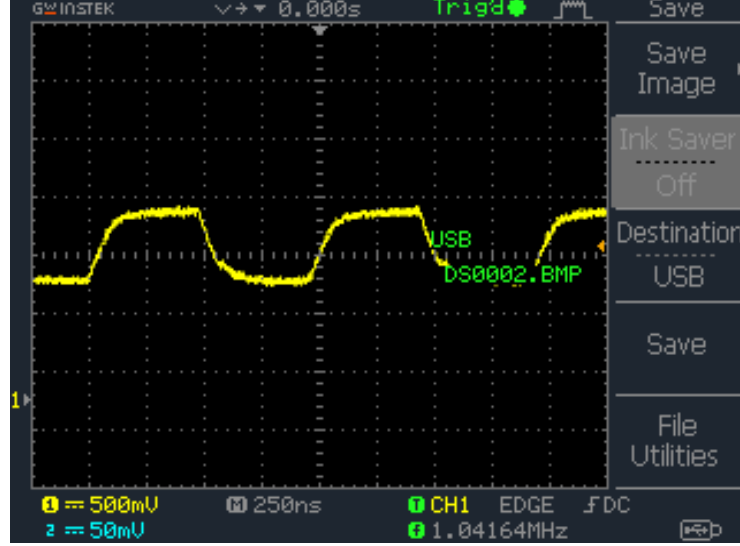
Şekil 6.3 10 Adet Veri ile Sinüs Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 25 değerine ulaştığında FPGA üzerinde binary cinsinden 25’e karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 4 veri aktarılmaktadır. 4 adet veri ile Şekil 6.4’de belirtildiği gibi sinüs sinyali oluşmaktadır. Frekans değeri 520,816 KHz olarak elde edilmektedir.



Şekil 6.4 4 Adet Veri ile Sinüs Dalga Formu

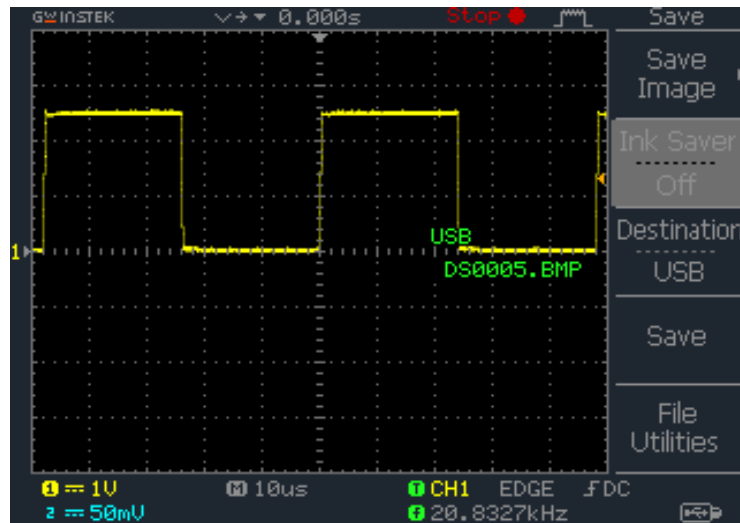
“send_index_value” sinyalinin değeri butonlar ile artırılarak 50 değerine ulaştığında FPGA üzerinde binary cinsinden 50’ye karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 2 veri aktarılmaktadır. 2 adet veri ile Şekil 6.5’de belirtildiği gibi sinüs sinyali oluşmaktadır. Frekans değeri 1,04164 MHz olarak elde edilmektedir.



Şekil 6.5 2 Adet Veri ile Sinüs Dalga Formu

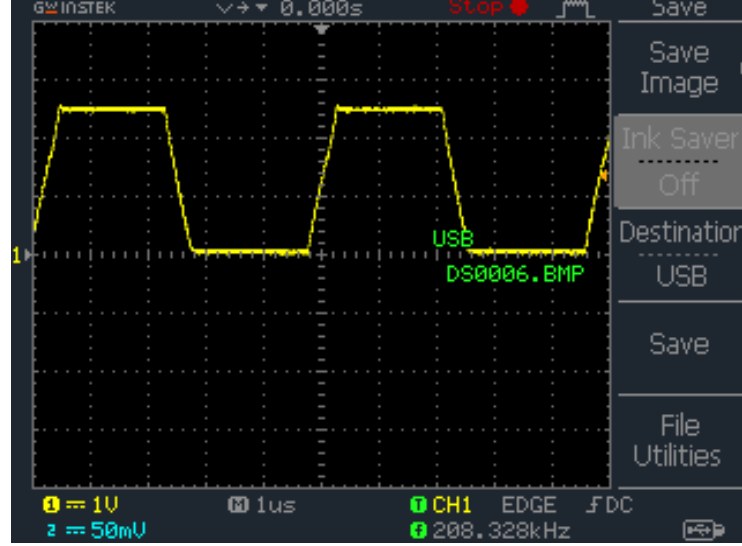
6.4.2. Kare Dalga Formu

Matlab’da verilerin oluşturulması esnasında Kare sinyal tipi seçilerek oluşturulmaktadır. Oluşturulan 100 adet veri FPGA’ye aktarılarak kare dalga oluşturulmaktadır. Butonlar ile kontrolü sağlanan “send_index_value” sinyalinin değeri artırılarak 1 olarak ayarlandığında 100 adet verinin tamamı çıkışa aktarılmaktadır. 100 adet veri ile Şekil 6.6’da belirtildiği gibi kare dalga oluşmaktadır. Frekans değeri 20,8327 KHz olarak elde edilmiştir.



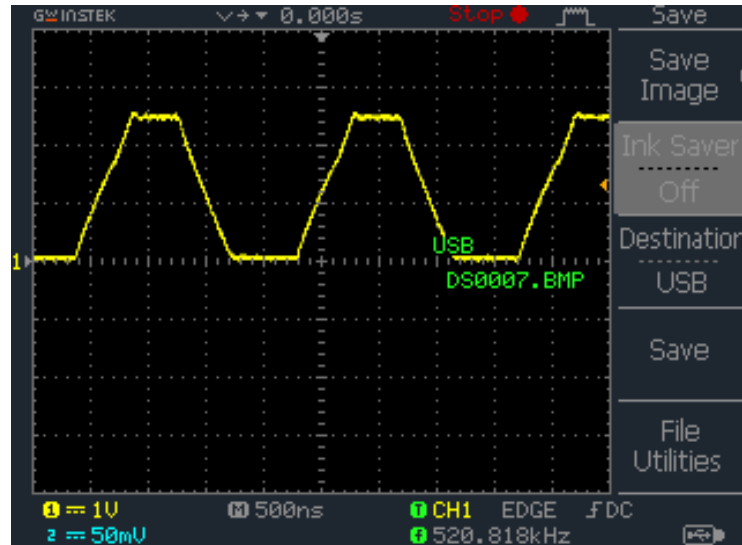
Şekil 6.6 100 Adet Veri ile Kare Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 10 değerine ulaştığında FPGA üzerinde binary cinsinden 10’a karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 10 veri aktarılmaktadır. 10 adet veri ile Şekil 6.7’de belirtildiği gibi kare dalga oluşmaktadır. Frekans değeri 208,328 KHz olarak elde edilmektedir.



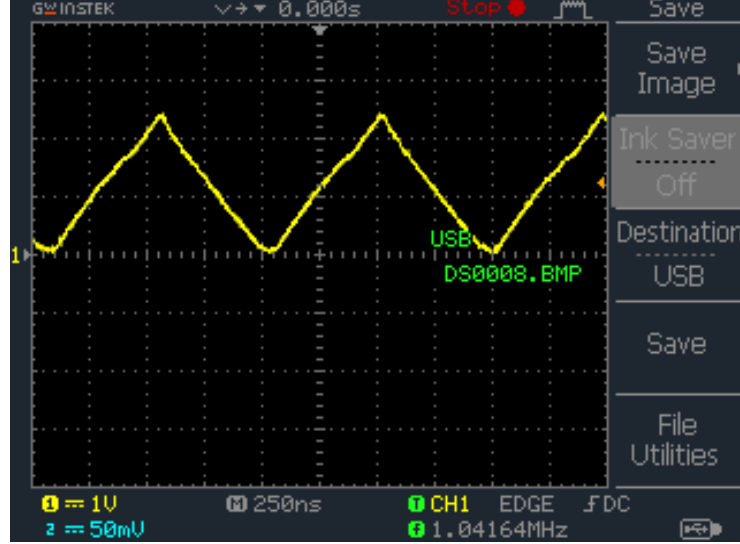
Şekil 6.7 10 Adet Veri ile Kare Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 25 değerine ulaştığında FPGA üzerinde binary cinsinden 25’e karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 4 veri aktarılmaktadır. 4 adet veri ile Şekil 6.8’de belirtildiği gibi kare dalga oluşmaktadır. Frekans değeri 520,818 KHz olarak elde edilmektedir.



Şekil 6.8 4 Adet Veri ile Kare Dalga Formu

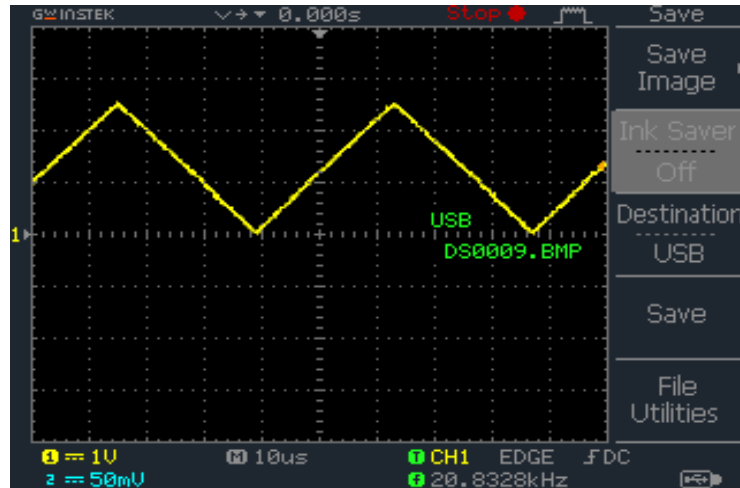
“send_index_value” sinyalinin değeri butonlar ile artırılarak 50 değerine ulaştığında FPGA üzerinde binary cinsinden 50’ye karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 2 veri aktarılmaktadır. 2 adet veri ile Şekil 6.9’da belirtildiği gibi kare dalga oluşmaktadır. Frekans değeri 1,04164 MHz olarak elde edilmektedir.



Şekil 6.9 2 Adet Veri ile Kare Dalga Formu

6.4.3. Üçgen Dalga Formu

Matlab’da verilerin oluşturulması esnasında Üçgen sinyal tipi seçilerek oluşturulmaktadır. Oluşturulan 100 adet veri FPGA’e aktarılarak üçgen dalga oluşturulmaktadır. Butonlar ile kontrolü sağlanan ve aynı zamanda FPGA üzerindeki ledler aracılığıyla takibi yapılabilen “send_index_value” sinyalinin değeri artırılarak 1 olarak ayarlandığında 100 adet verinin tamamı çıkışa aktarılmaktadır. 100 adet veri ile Şekil 6.10’da belirtildiği gibi üçgen dalga oluşmaktadır. Frekans değeri 20,8328 KHz olarak elde edilmiştir.



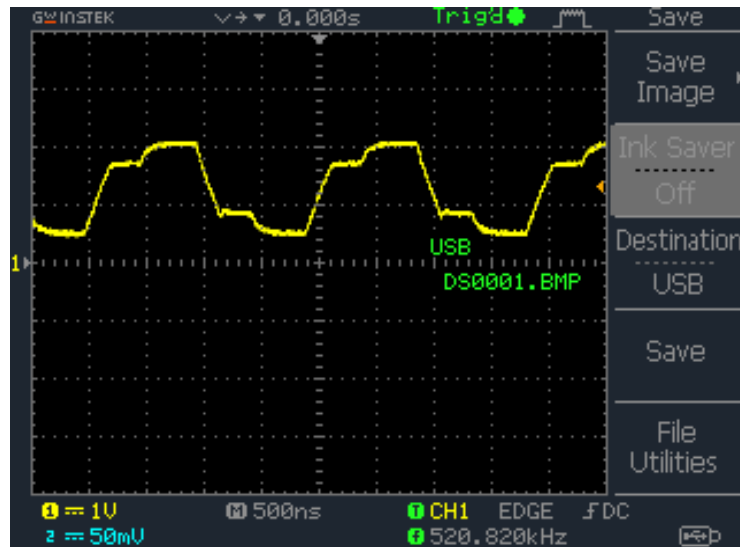
Şekil 6.10 100 Adet Veri ile Üçgen Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 10 değerine ulaştığında FPGA üzerinde binary cinsinden 10’a karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 10 veri aktarılmaktadır. 10 adet veri ile Şekil 6.11’de belirtildiği gibi üçgen dalga oluşmaktadır. Frekans değeri 208,328 KHz olarak elde edilmektedir.



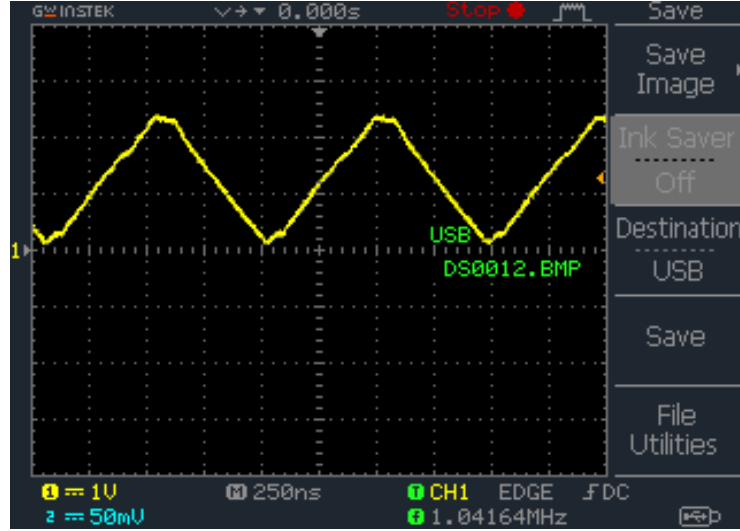
Şekil 6.11 10 Adet Veri ile Üçgen Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 25 değerine ulaştığında FPGA üzerinde binary cinsinden 25’e karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 4 veri aktarılmaktadır. 4 adet veri ile Şekil 6.12’de belirtil- diği gibi üçgen dalga oluşmaktadır. Frekans değeri 520,820 KHz olarak elde edilmektedir.



Şekil 6.12 4 Adet Veri ile Üçgen Dalga Formu

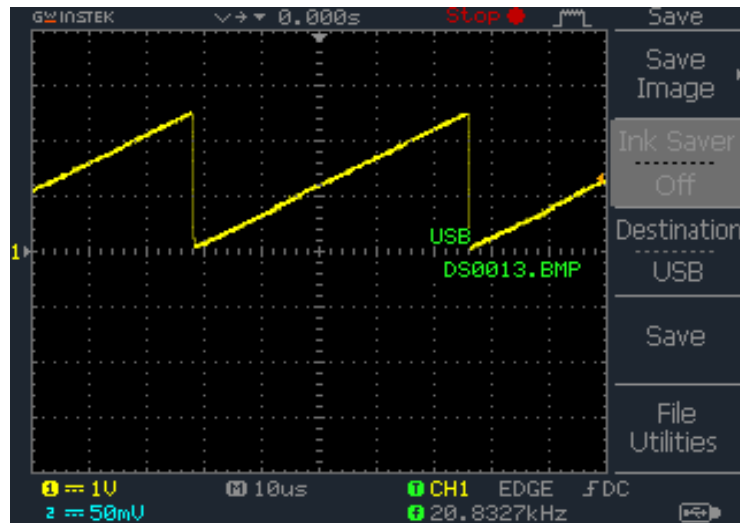
“send_index_value” sinyalinin değeri butonlar ile artırılarak 50 değerine ulaştığında FPGA üzerinde binary cinsinden 50’ye karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 2 veri aktarılmaktadır. 2 adet veri ile Şekil 6.13’de belirtildiği gibi üçgen dalga oluşmaktadır. Frekans değeri 1,04164 MHz olarak elde edilmektedir.



Şekil 6.13 2 Adet Veri ile Üçgen Dalga Formu

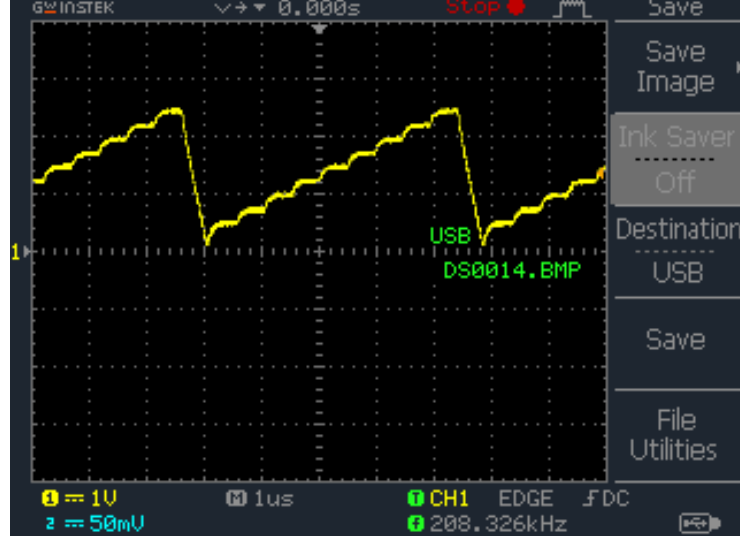
6.4.4. Testere Dişi Dalga Formu

Matlab’da verilerin oluşturulması esnasında Testere Dişi sinyal tipi seçilerek oluşturulmaktadır. Oluşturulan 100 adet veri FPGA’e aktarılarak testere dişi dalga oluşturulmaktadır. Butonlar ile kontrolü sağlanan ve aynı zamanda FPGA üzerindeki ledler aracılığıyla takibi yapılabilen “send_index_value” sinyalinin değeri artırılarak 1 olarak ayarlandığında 100 adet verinin tamamı çıkışa aktarılmaktadır. 100 adet veri ile Şekil 6.14’de belirtildiği gibi testere dişi dalga oluşmaktadır. Frekans değeri 20,8327 KHz olarak elde edilmiştir.



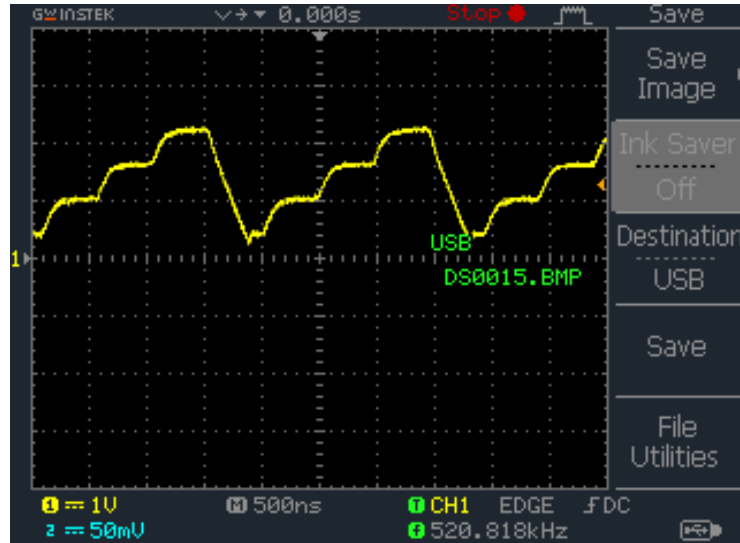
Şekil 6.14 100 Adet Veri ile Testere Dişi Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 10 değerine ulaştığında FPGA üzerinde binary cinsinden 10’a karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 10 veri aktarılmaktadır. 10 adet veri ile Şekil 6.15’de belirtildiği gibi testere dişi dalga oluşmaktadır. Frekans değeri 208,326 KHz olarak elde edilmektedir.



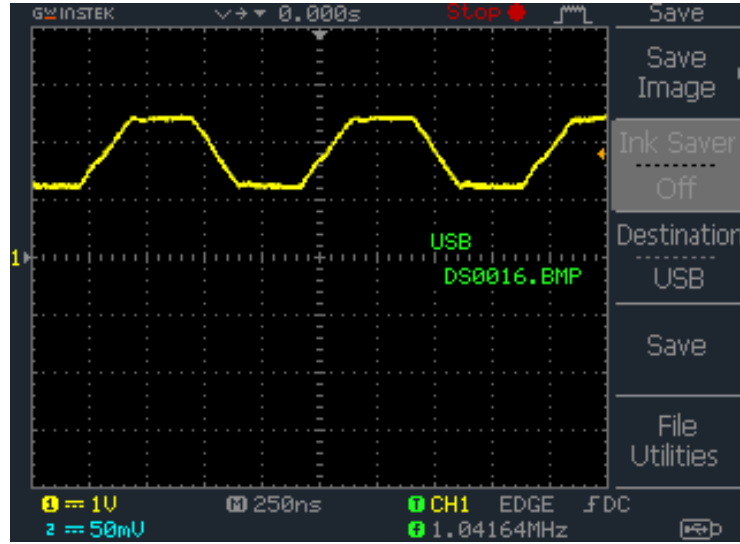
Şekil 6.15 10 Adet Veri ile Testere Dişi Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 25 değerine ulaştığında FPGA üzerinde binary cinsinden 25’e karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 4 veri aktarılmaktadır. 4 adet veri ile Şekil 6.16’da belirtildiği gibi testere dişi dalga oluşmaktadır. Frekans değeri 520,818 KHz olarak elde edilmektedir.



Şekil 6.16 4 Adet Veri ile Testere Dişi Dalga Formu

“send_index_value” sinyalinin değeri butonlar ile artırılarak 50 değerine ulaştığında FPGA üzerinde binary cinsinden 50’ye karşılık gelen ledlerin yandığı görülebilmektedir. Bu durumda 100 adet veriden sadece 2 veri aktarılmaktadır. 2 adet veri ile Şekil 6.17’de belirtildiği gibi testere dişi dalga oluşmaktadır. Frekans değeri 1,04164 MHz olarak elde edilmektedir.



Şekil 6.17 2 Adet Veri ile Testere Dişi Dalga Formu

BÖLÜM 7

7. SONUÇ VE ÖNERİLER

Bu lisans tezi kapsamında fonksiyon üretici için gerekli yazılım ve donanımlar detaylı tanımlanmıştır. Öncelikle bilgisayardan FPGA’ye veri aktarımı için gerekli olan UART protokolü tasarlanmıştır. Daha sonrasında FPGA’den DAC (Digital-Analog Converter)’a veri aktarımı için SPI protokolü tasarlanmıştır. Tasarlanan protokollerin kontrolleri, aktarım uygulamalarının sağlandığı bir Üst Modül (Top Modul) tasarlanmıştır. Üst modülde bilgisayardan gelen 8 bitlik verileri sırasıyla çekerek 16 bit olarak kaydetmektedir. Kayıt işleminden sonra DAC’a gönderilmektedir.

Tasarlanan bu süreç sonunda istenilen sinyal tipine ait sinyal katsayı verilerinin gönderilmesi doğrultusunda farklı sinyal tipleri üretilebilmektedir. Aynı zamanda gönderilen veri sayısının kontrolünün sağlanmasıyla Frekans değeri de kontrol edilebilmektedir.

İstenilen farklı sinyal tipleri başarıyla üretilebilmiştir ancak proje kapsamında hedeflenen 1 MHz frekans değerine ulaşıldığında üretilen sinyal dalga formlarının düzensizleştiği gözlemlenmiştir. Sinyal katsayı verilerinin seri olarak gönderilmesinden kaynaklı olarak tek bir örnek veride elde edilebilen maksimum frekans değeri 2 MHz’dir. Dolayısıyla 1 MHz üzeri frekans değerleri istendiğinde dalga formu istenilen şekilde elde edilememektedir. Düzgün dalga formlarında sinyal üretebilmek için sinyal katsayı verilerinin fazla olması gerekmektedir. Böylece frekans değeri azalmaktadır.

Bu soruna çözüm olarak veri gönderme işlemleri geliştirilerek seri veri iletimi yerine paralel veri iletimi yapılabilir. Böylece 1 MHz ve üzerinde istenilen frekans değerleri rahatlıkla elde edilebilecektir.

KAYNAKLAR

1. Vivado, https://en.wikipedia.org/wiki/Vivado#cite_note-hls1-6 (Eriřim Tarihi: Nisan, 2024).
2. Xilinx Vivado, Design Entry & Implementation, <https://www.xilinx.com/products/designtools/vivado/implementation.html#synthesis> (Eriřim Tarihi: Nisan, 2024)
3. PmodDA3 Reference Manual, “Datasheet”, https://digilent.com/reference/_media/reference/pmod/pmodda3/pmodda3_rm.pdf (Eriřim Tarihi: Kasım, 2023)
4. Notepad++, <https://tr.wikipedia.org/wiki/Notepad%2B%2B> (Eriřim Tarihi: Aralık, 2023)
5. MATLAB (matrix laboratory), <https://tr.wikipedia.org/wiki/MATLAB> (Eriřim Tarihi: Mart, 2024)
6. Analog Devices, AD5541A, “Datasheet”, AD5541A.pdf (analog.com), 1–20, 2010.
7. Basys 3 FPGA Board Reference Manual, basys3_rm.pdf (digilent.com), “Datasheet”, 1-19, 2019.
8. Kulaçoğlu, Ş., Dinç, M. F., “FPGA ile Sayısal Ön Bozumlama”, Lisans Tezi, Karabük Üniversitesi Mühendislik Fakültesi, Karabük, 18-28 (2015).
9. Aydoğan, N. M., “FPGA Kullanılarak Doğrudan Sayısal Sentez Tabanlı Fonksiyon Üretici Tasarımı”, Yüksek Lisans Tezi, Hacettepe Üniversitesi Fen Bilimleri Enstitüsü, Ankara, 2013.
10. Al-Bokhaiti, M. K., “FPGA Based Frequency Measurement for The Purpose of Synchronization”, Communications Laboratory – University of Kassel, 1-10, 2010.

11. Çetinkaya B., Işık B., “FPGA Üzerinde Sinyal İşleme Uygulamaları”, OPTOEL Mühendislik, 2021.
12. Aykenar M. B., https://github.com/mbaykenar/apis_anatolia, “VHDL ile FPGA PROGRAMLAMA”, Ders 18, (Erişim Tarihi: Aralık, 2023)
13. Çavuşlu M. A., Kösten M. M., “VHDL ile Sayısal Tasarım ve FPGA Uygulamaları”, KODLAB, 2017.

EKLER

EK 1. UART_RX Modülü

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity uart_rx is
generic (
c_clkfreq      : integer := 100_000_000;
c_baudrate     : integer := 9_600
);
port (
clk            : in std_logic;
rx_i           : in std_logic;
dout_o         : out std_logic_vector (7 downto 0);
rx_done_tick_o : out std_logic
);
end uart_rx;
```

architecture Behavioral of uart_rx is

```
constant c_bittimerlim : integer := c_clkfreq/c_baudrate;
type states is (S_IDLE,S_START,S_DATA,S_STOP);
signal state : states := S_IDLE;

signal bittimer : integer range 0 to c_bittimerlim :=0; -- T kadar sureyi sayar
signal bitcntr  : integer range 0 to 7 :=0;           -- Kac bit orneklendigini sayar
signal shreg    : std_logic_vector (7 downto 0) := (others => '0');
```

```

begin
P_MAIN: process (clk) begin
if rising_edge(clk) then
    case state is

        when S_IDLE =>
            rx_done_tick_o <='0';
            bittimer        <= 0;
            if(rx_i='0') then
                state <= S_START;
            end if;

        when S_START =>
            if (bittimer = c_bittimerlim/2-1) then
                state <= S_DATA;
                bittimer <= 0;
            else
                bittimer <= bittimer + 1;
            end if;

        when S_DATA =>
            if (bittimer = c_bittimerlim-1) then
                if (bitcnt = 7) then
                    state <= S_STOP;
                    bitcnt <= 0;
                else
                    bitcnt<= bitcnt + 1;
                end if;
                shreg <= rx_i & (shreg(7 downto 1));
                bittimer <=0;
            else
                bittimer <= bittimer +1;
            end if;
    end case;
end if;

```

```

        when S_STOP =>
--rx_done_tick_o <= '1'; buraya yazilmis olsaydi stop'a gelir gelmez T kadar sure bekleme-
den veri hazir olacakti.
            if(bittimer = c_bittimerlim-1) then
                state          <= S_IDLE;
                bittimer        <= 0;
                rx_done_tick_o <= '1';
            else
                bittimer <= bittimer + 1;
            end if;

        end case;

    end if;
end process P_MAIN;

dout_o <= shreg;

end Behavioral;

```

EK 2. SPI Modülü

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

entity spi_module is

```
Port (
    clk : in  STD_LOGIC;
    data : in  STD_LOGIC_VECTOR (15 downto 0);
    sclk : out STD_LOGIC;
    din : out STD_LOGIC;
    ldac : out STD_LOGIC;
    cs : out STD_LOGIC);
end spi_module;
```

architecture Behavioral of spi_module is

```
    signal data_i : STD_LOGIC_VECTOR(15 downto 0); -- Gonderilecek veriyi saklamak
    icin bir sinyal
```

```
    signal run_stop : STD_LOGIC_VECTOR (1 downto 0):=(others=> '0');
    -- Clock Divider degiskenleri
    constant CLOCK_DIVIDER : integer := 1; -- Saat bolucu degeri
    signal clk_div_counter : integer range 0 to CLOCK_DIVIDER - 1 := 0;
    signal clk_divided : std_logic := '0';
    signal set : integer range 0 to 23 := 0;
    signal il: integer range 0 to 16 := 16;
begin
    data_i <= data;
```

```

process(clk)
begin
    if rising_edge(clk) then
        if clk_div_counter = CLOCK_DIVIDER - 1 then
            clk_div_counter <= 0;
            clk_divided <= not clk_divided; -- Ters cevirilir, boylece saat sinyali dusen kenar-
larinda degisir.
        else
            clk_div_counter <= clk_div_counter + 1;
        end if;

        -- Saat sinyalinin atanmasi
        sclk <= clk_divided;

        if clk_divided = '1' then
            if run_stop (0) <= '0' then
                run_stop (0) <= '1';

                case set is
                when 0 =>
                    cs <= '1';
                    ldac <= '1';
                when 1 =>
                    cs <= '1';
                    ldac <= '1';
                when 2 =>
                    cs <= '1';
                    ldac <= '1';
                when 3 =>
                    cs <= '0';
                    il <= 15;

```

```

when 20 =>
    cs <= '1';
    ldac <= '1';
when 21 to 22 =>
    cs <= '1';
    ldac <= '0';
when 23 =>
    cs <= '1';
    ldac <= '1';
when 4 to 19 =>
    ldac <= '1';
    din <= data_i(i1);
    i1 <= (i1-1);
end case;
    set <= (set + 1) mod 24;
end if;
else
    run_stop (0) <= '0';
end if;
end if;
end process;

end Behavioral;

```

EK 3. Üst Modül

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;

entity top_module is
generic (
    c_clkfreq    : integer := 100_000_000;
    c_baudrate    : integer := 9_600
);

    Port (
        clk      : in STD_LOGIC;
        rx_i     : in STD_LOGIC;
        sclk     : out STD_LOGIC;
        din      : out STD_LOGIC;
        ldac     : out STD_LOGIC;
        cs       : out STD_LOGIC;
        leds_o   : out STD_LOGIC_VECTOR (15 downto 0);
        btnU     : in STD_LOGIC:= '0';
        btnD     : in STD_LOGIC:= '0'
    );

end top_module;
```


architecture Behavioral of top_module is

component spi_module is

```
Port (  
    clk : in STD_LOGIC;  
    data : in STD_LOGIC_VECTOR(15 downto 0);  
    sclk : out STD_LOGIC;  
    din : out STD_LOGIC;  
    ldac : out STD_LOGIC;  
    cs : out STD_LOGIC  
);  
end component;
```

component uart_rx is

```
generic (  
    c_clkfreq : integer := 100_000_000;  
    c_baudrate : integer := 9_600  
);  
port (  
    clk : in std_logic;  
    rx_i : in std_logic;  
    dout_o : out std_logic_vector (7 downto 0);  
    rx_done_tick_o : out std_logic  
);  
end component;
```

type data_array is array (0 to 99) of std_logic_vector(15 downto 0);

signal data_buffer : data_array := (others => (others => '0'));

signal buffer_index : integer := 0;

signal rx_done_count : integer := 0;

signal send_index : integer range 0 to data_array'length - 1 := 0;

signal send_data : std_logic := '0';

signal current_data : std_logic_vector(15 downto 0) := (others => '0');

```

signal led : std_logic_vector(15 downto 0) := (others => '0');
signal dout : std_logic_vector(7 downto 0) := (others => '0');
signal rx_done_tick : std_logic := '0';
signal cs_i : std_logic;
signal l_dac_i : std_logic;
signal send_index_value: integer:= 0;

signal btnU_prev : std_logic := '0'; -- Baslangicta buton basilmamis kabul edilir
signal btnD_prev : std_logic := '0'; -- Baslangicta buton basilmamis kabul edilir
begin

i_uart_rx : uart_rx
    generic map (
        c_clkfreq => c_clkfreq,
        c_baudrate => c_baudrate
    )
    port map (
        clk => clk,
        rx_i => rx_i,
        dout_o => dout,
        rx_done_tick_o => rx_done_tick
    );

i_spi_module : spi_module
    port map (
        clk => clk,
        sclk => sclk,
        din => din,
        ldac => l_dac_i,
        cs => cs_i,
        data => current_data
    );

```

```

process(clk)
begin
    if rising_edge(clk) then
        if rx_done_tick = '1' then
            if rx_done_count = 0 then
                data_buffer(buffer_index)(15 downto 8) <= dout;
                rx_done_count <= 1;
            elsif rx_done_count = 1 then
                data_buffer(buffer_index)(7 downto 0) <= dout;
                rx_done_count <= 0;
                buffer_index <= buffer_index + 1;
            end if;
        end if;

        if buffer_index = 100 then
            send_data <= '1';
        end if;

    end if;
end process;

process(clk)
begin
    if rising_edge(clk) then
        if btnU = '1' and btnU_prev = '0' then
            if send_index_value < 50 then
                send_index_value <= send_index_value + 1;
            end if;
            btnU_prev <= '1';
        elsif btnU = '0' and btnU_prev = '1' then
            btnU_prev <= '0'; -- Buton basildiginda durumu '1' olarak ayarla
        end if;
    end if;
end process;

```

```

        if btnD = '1' and btnD_prev = '0' then
            if send_index_value > 0 then
                send_index_value <= send_index_value - 1;
            end if;
            btnD_prev <= '1'; -- Buton durumunu guncelle
        elsif btnD = '0' and btnD_prev = '1' then
            btnD_prev <= '0';
        end if;
        led <= std_logic_vector(to_unsigned(send_index_value, leds_o'length));
    end if;
end process;

process(sclk)
begin

    if rising_edge(l_dac_i) then

        if send_data = '1' then

            current_data <= data_buffer(send_index);
            send_index <= 0 mod data_array'length;
            send_index <= (send_index + send_index_value) mod data_array'length;

        end if;
    end if;
end process;

cs <= cs_i;
ldac <= l_dac_i;
leds_o <= led;

end Behavioral;

```

EK 4. Matlab Yazılımı

```
function create_signal_file()
    % Kullanıcı Arayüzü Oluştur
    fig = uifigure('Name', 'Sinyal Oluşturucu', 'Position', [100 100 850 400]);
    dropdownSignalType = uidropdown(fig, 'Items', {'Sinüs', 'Kare', 'Üçgen', 'Testere'}, 'Position', [20, 350, 100, 22]);
    spinner = uispinner(fig, 'Position', [130, 350, 100, 22], 'Value', 3, 'Limits', [1, 10000]);
    % Değer başlangıçta 3 olarak ayarlandı
    dropdownFormat = uidropdown(fig, 'Items', {'Hex', 'Dec'}, 'Position', [240, 350, 100, 22]); % Format seçimi ekleniyor
    dropdownBitDepth = uidropdown(fig, 'Items', {'8 Bit', '16 Bit'}, 'Position', [350, 350, 100, 22]); % Bit derinliği seçimi ekleniyor
    btn = uibutton(fig, 'Position', [460, 350, 150, 22], 'Text', 'Oluştur ve COE Kaydet', 'ButtonPushedFcn', @(btn,event) generateSignal(dropdownSignalType.Value, spinner.Value, dropdownFormat.Value, dropdownBitDepth.Value, fig));
    % Grafik Alanı Başlangıçta Oluşturulur
    ax = uiaxes('Parent', fig, 'Position', [50 50 750 250]); % UIAxes oluştur
end

function generateSignal(type, n, format, bitDepth, fig)
    % Mevcut Grafik Verilerini Temizle
    ax = findobj(fig, 'Type', 'Axes');
    cla(ax); % Mevcut çizimleri temizle

    t = linspace(0, 2*pi, n+1); % n+1 nokta oluştur
    t = t(1:end-1); % Son noktayı çıkartarak n nokta elde et

    switch type
        case 'Sinüs'
            y = sin(t);
        case 'Kare'
            y = square(t);
```

```

    case 'Üçgen'
        y = sawtooth(t, 0.5); % 0.5 üçgen dalga için görev oranı
    case 'Testere'
        y = sawtooth(t); % Testere dişi dalga
    end

    % Dalga şeklini grafik üzerinde göster
    plot(ax, t, y);
    title(ax, [type ' Dalga Şekli']);
    xlabel(ax, 'Zaman (rad)');
    ylabel(ax, 'Amplitüd');

    % Verileri COE dosyasına kaydet
    writeCOEFile(y, format, bitDepth, [type '_signal.coe']);
end

function writeCOEFile(data, format, bitDepth, filename)
    % Verileri COE formatında kaydet
    fid = fopen(filename, 'w');

    if strcmp(format, 'Hex')
        fprintf(fid, 'memory_initialization_radix=16;\n');
    else
        fprintf(fid, 'memory_initialization_radix=10;\n');
    end
    fprintf(fid, 'memory_initialization_vector=\n');

    if strcmp(bitDepth, '16 Bit')
        data_scaled = floor((data + 1) * 32767.5); % -1 ile 1 arasını 0 ile 65535 aralığına dö-
nüştür
        max_val = 65535;
    else
        data_scaled = floor((data + 1) * 127.5); % -1 ile 1 arasını 0 ile 255 aralığına dönüştür

```

```

    max_val = 255;
end
data_scaled = min(max(data_scaled, 0), max_val); % 0 ile max_val arasına sınırla

for i = 1:length(data_scaled)
    if strcmp(format, 'Hex')
        if strcmp(bitDepth, '16 Bit')
            str = sprintf('x"%04X"', data_scaled(i)); % Veriyi x"FFFF" formatında yaz
        else
            str = sprintf('x"%02X"', data_scaled(i)); % Veriyi x"FF" formatında yaz
        end
    else
        str = num2str(data_scaled(i));
    end

    if i == length(data_scaled)
        fprintf(fid, '%s;\n', str); % Son satırdaki noktalı virgül
    else
        fprintf(fid, '%s, ', str); % Her veri ögesini yan yana yaz
    end
end

fclose(fid);
disp(['COE dosyası kaydedildi: ' filename]);
end

```

EK 5. SPI ile Veri Gönderme Modülü

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

entity spi_module is

Port (

```
    clk : in  STD_LOGIC;
    sclk : out STD_LOGIC;
    din  : out STD_LOGIC;
    ldac : out STD_LOGIC;
    cs   : out STD_LOGIC);
```

end spi_module;

architecture Behavioral of spi_module is

```
    type data_array_type is array (0 to 3) of STD_LOGIC_VECTOR(15 downto 0);
    signal data_array : data_array_type := (x"7FFF", x"FFFF", x"7FFF", x"0000");
```

```
    signal set_index : integer range 0 to data_array'length - 1 := 0;
```

-- Baslangicta indeks 0 olarak ayarlanir

```
    signal data_i : STD_LOGIC_VECTOR(15 downto 0);
```

-- Gonderilecek veriyi saklamak icin bir sinyal

```
    signal run_stop : STD_LOGIC_VECTOR (1 downto 0):=(others=> '0');
```

-- Clock Divider degiskenleri

```
    constant CLOCK_DIVIDER : integer := 1; -- Saat bolucu degeri
```

```
    signal clk_div_counter : integer range 0 to CLOCK_DIVIDER - 1 := 0;
```

```
    signal clk_divided : std_logic := '0';
```

```
    signal set : integer range 0 to 23 := 0;
```

```
    signal i1: integer range 0 to 16 := 16;
```



```

begin
    data_i <= data_array(set_index);
process(clk)
begin
    if rising_edge(clk) then

        if clk_div_counter = CLOCK_DIVIDER - 1 then
            clk_div_counter <= 0;
            clk_divided <= not clk_divided; -- Ters çevirilir, böylece saat sinyali dusen kenar-
larinda degisir.
        else
            clk_div_counter <= clk_div_counter + 1;
        end if;

        -- Saat sinyalinin atanmasi
        sclk <= clk_divided;

        if clk_divided = '1' then
            if run_stop (0) <= '0' then
                run_stop (0) <= '1';

                case set is
                    when 0 =>
                        cs <= '1';
                        ldac <= '1';
                    when 1 =>
                        cs <= '1';
                        ldac <= '1';
                    when 2 =>
                        cs <= '1';
                        ldac <= '1';

```

```

when 3 =>
    cs <= '0';
    i1 <= 15;
when 20 =>
    cs <= '1';
    ldac <= '1';
when 21 to 22 =>
    cs <= '1';
    ldac <= '0';
when 23 =>
    cs <= '1';
    ldac <= '1';
    set_index <= (set_index + 1) mod data_array'length;
when 4 to 19 =>
    ldac <= '1';
    din <= data_i(i1);
    i1 <= (i1-1);
end case;
set <= (set + 1) mod 24;
end if;
else
    run_stop (0) <= '0';
end if;
end if;
end process;

end Behavioral;

```

EK 6. TestBench Yazılımı

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity tb_top_module is
generic (
c_clkfreq      : integer := 100_000_000;
c_baudrate      : integer := 9_600
);
end tb_top_module;

architecture Behavioral of tb_top_module is

component top_module is
generic (
c_clkfreq      : integer := 100_000_000;
c_baudrate      : integer := 9_600
);

Port (
clk      : in STD_LOGIC;
rx_i     : in STD_LOGIC;
sclk     : out STD_LOGIC;
din      : out STD_LOGIC;
ldac     : out STD_LOGIC;
cs       : out STD_LOGIC;
leds_o   : out STD_LOGIC_VECTOR (15 downto 0)
);

end component;

end component;
```

```

signal clk   : std_logic;
signal rx_i  : std_logic;
signal sclk  : std_logic;
signal din   : std_logic;
signal ldac  : std_logic;
signal cs    : std_logic;
signal leds_o : STD_LOGIC_VECTOR(15 downto 0);

constant c_baud : time := 104166 ns;
constant c_hex1  : std_logic_vector (9 downto 0) := '1' & x"23" & '0';
constant c_hex2  : std_logic_vector (9 downto 0) := '1' & x"2F" & '0';
constant c_hex3  : std_logic_vector (9 downto 0) := '1' & x"15" & '0';
constant c_hex4  : std_logic_vector (9 downto 0) := '1' & x"a8" & '0';

constant c_hex5  : std_logic_vector (9 downto 0) := '1' & x"AA" & '0';
constant c_hex6  : std_logic_vector (9 downto 0) := '1' & x"FF" & '0';
-- Clock period definition
constant clk_period : time := 10 ns; -- 100 MHz
begin

top_le : top_module

Port map (
    clk  => clk   ,
    rx_i => rx_i  ,
    sclk => sclk  ,
    din  => din   ,
    ldac => ldac  ,
    cs   => cs    ,
    leds_o => leds_o
);

```

```

-- clock process definitions
clk_process : process
begin
    clk <= '1';
    wait for clk_period/2;
    clk <= '0';
    wait for clk_period/2;
end process clk_process;

test_pro : process
begin
wait for clk_period*10;
for i in 0 to 9 loop
    rx_i <= c_hex1(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex2(i);
    wait for c_baud;
end loop;
wait for 20 us;
for i in 0 to 9 loop
    rx_i <= c_hex3(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex4(i);
    wait for c_baud;
end loop;

```

```

wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex1(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex2(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex3(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex4(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex5(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex6(i);
    wait for c_baud;
end loop;

```

```

wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex1(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex2(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex3(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex4(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex5(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex1(i);
    wait for c_baud;
end loop;

```

```

wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex2(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex3(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex4(i);
    wait for c_baud;
end loop;
wait for 10 us;
for i in 0 to 9 loop
    rx_i <= c_hex5(i);
    wait for c_baud;
end loop;
wait;
assert false
report "SIM DONE"
severity failure;

end process;

end Behavioral;

```


ÖZGEÇMİŞ

Kişisel Bilgiler

Soyadı, adı : ALTURAN Salim Enes
Uyruğu : Türkiye Cumhuriyeti
Doğum tarihi ve yeri : 31.07.2001- Karabük
Telefon : +90 536 296 7897
e-mail : salimenesalturan29@gmail.com

Eğitim

Derece	Eğitim Birimi	Mezuniyet Tarihi
Lise	Safranbolu Mesleki ve Teknik Anadolu Lisesi	2019

Yabancı Dil

İngilizce

Kişisel Bilgiler

Soyadı, adı : KARAMAN Muzaffer
Uyruğu : Türkiye Cumhuriyeti
Doğum tarihi ve yeri : 05.05.2000- Ankara
Telefon : +90 551 057 3585
e-mail : muzafferkaraman06@gmail.com

Eğitim

Derece	Eğitim Birimi	Mezuniyet Tarihi
Lise	Çankaya Lisesi	2018

Yabancı Dil

İngilizce